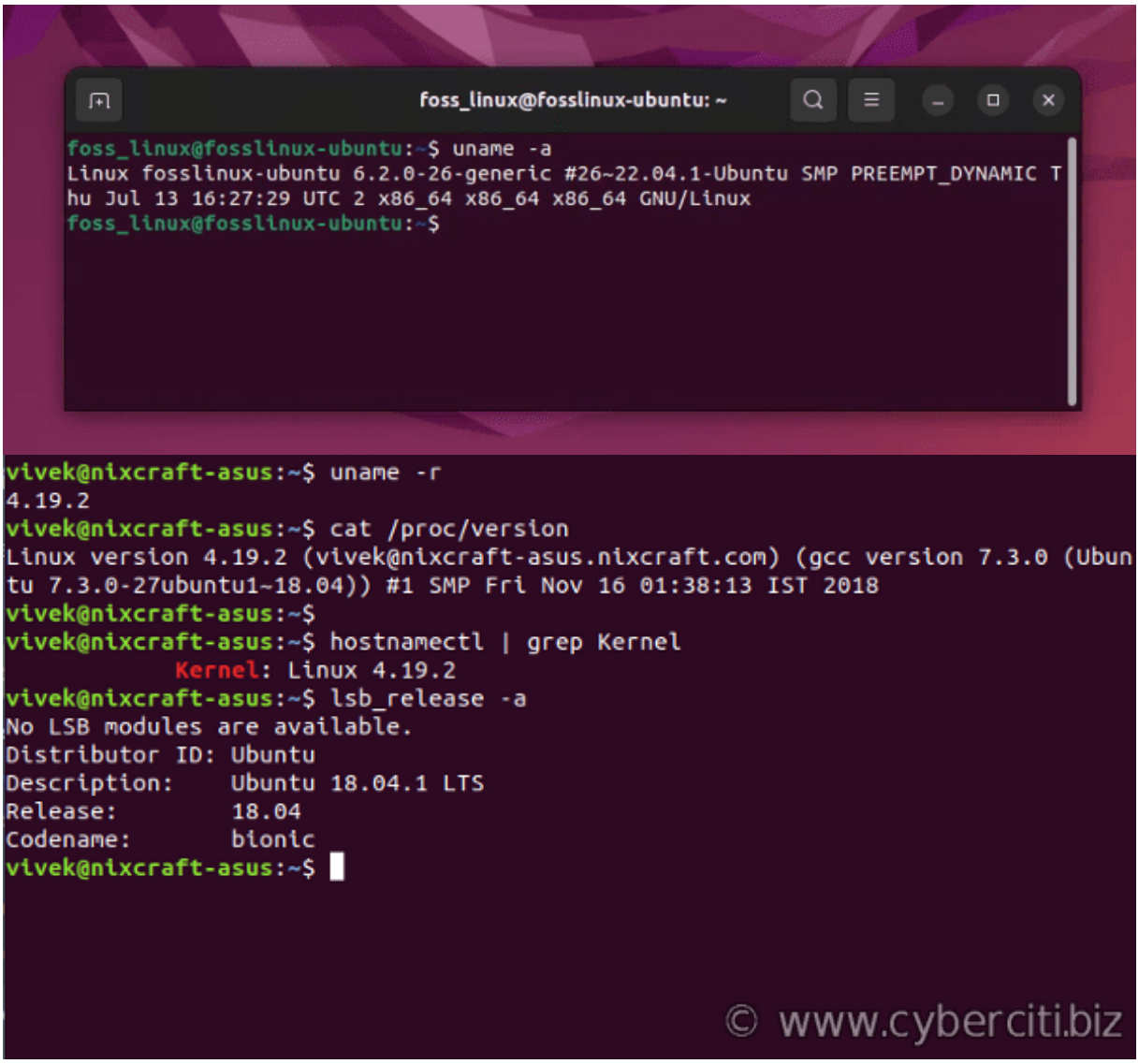
Mise à jour du noyau Linux

Étape 1 : Vérifier la version actuelle du noyau

Ouvre un terminal et exécute :

```
uname -r
```

➡️ Cela affiche la version du noyau actuellement utilisée.



🔄 Étape 2 : Mettre à jour la liste des paquets

Avant toute mise à jour, il faut synchroniser les dépôts :

```
sudo apt update
```

⬆️ Étape 3 : Mettre à jour le noyau (méthode standard – recommandée)

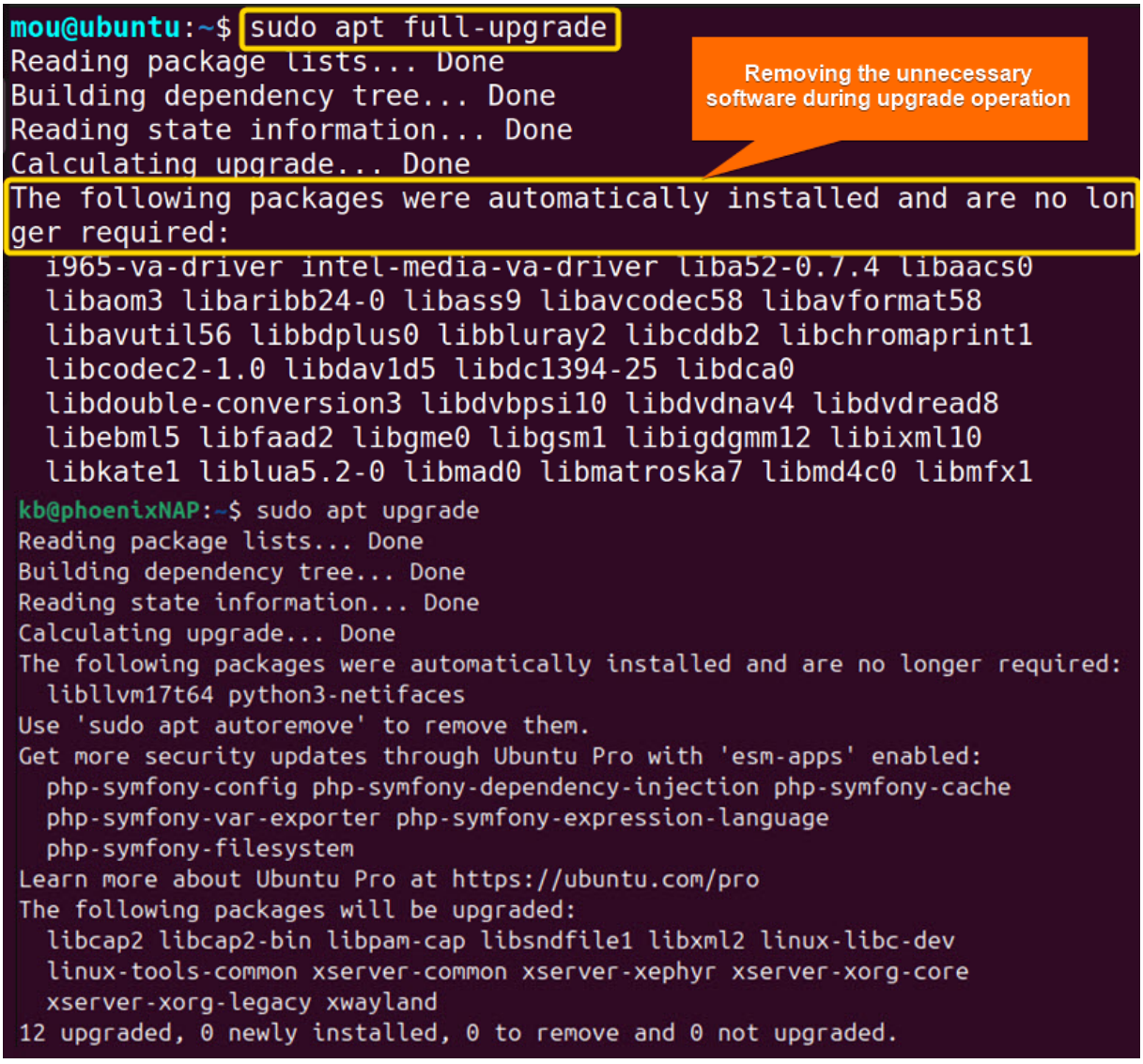
Cette méthode est **sûre et officielle** (kernel fourni par Ubuntu).

Commande :

```
sudo apt full-upgrade
```

📌 Cette commande :

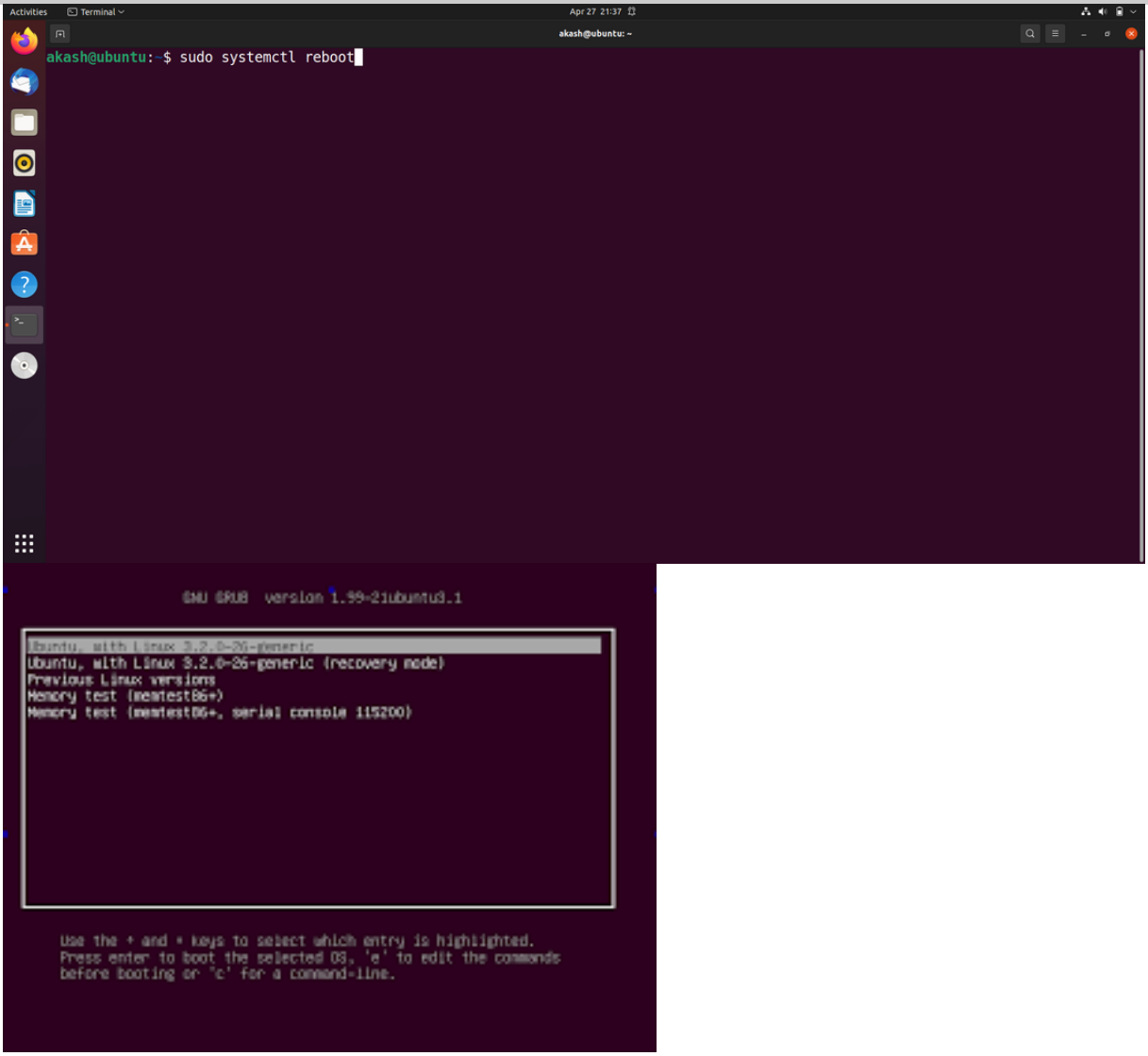
- installe les nouvelles versions du noyau
- gère correctement les dépendances



🔁 Étape 4 : Redémarrer le système

Le nouveau noyau **ne sera actif qu'après redémarrage** :

```
sudo reboot
```

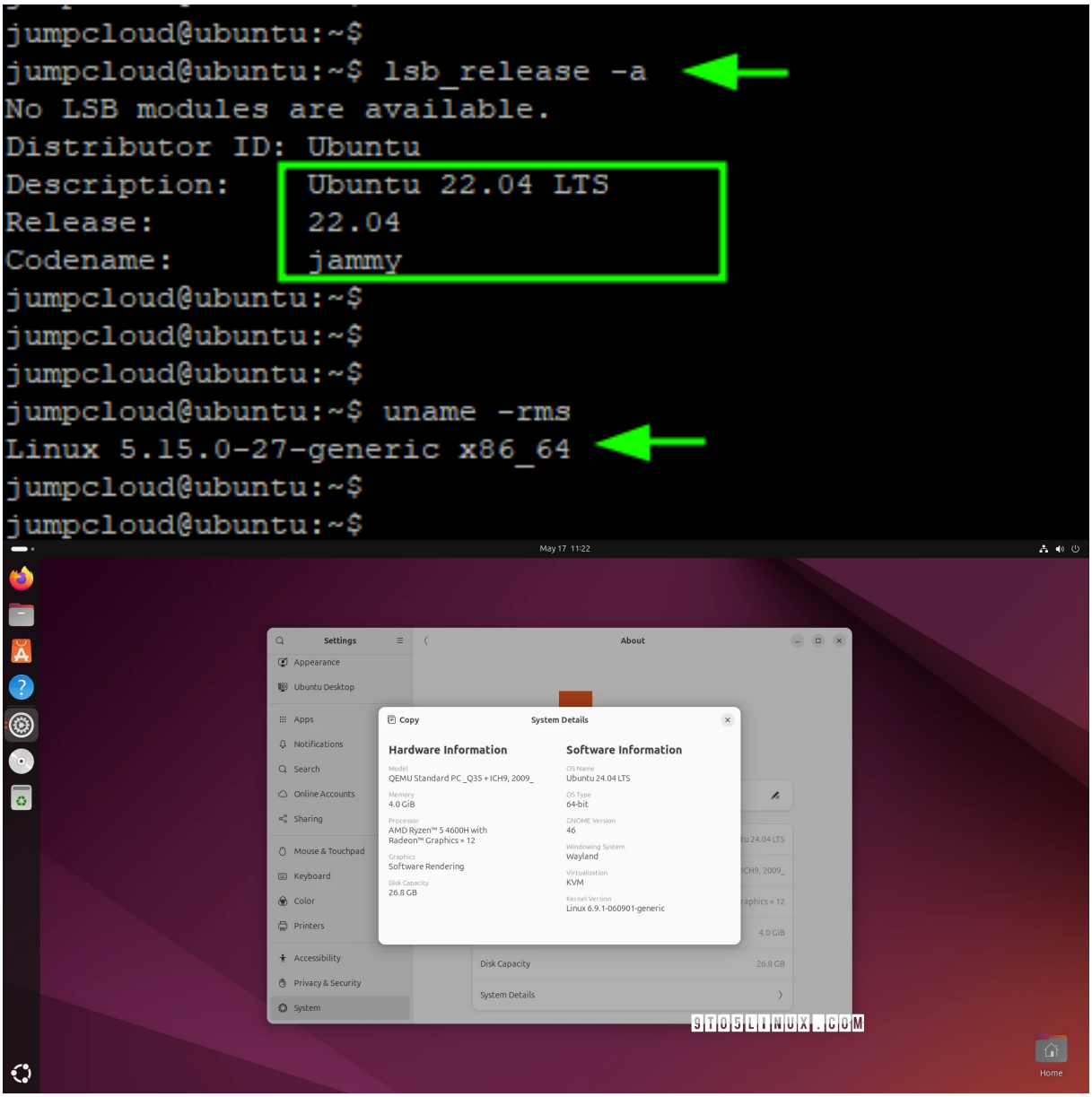


✅ Étape 5 : Vérifier que le nouveau noyau est actif

Après redémarrage :

```
uname -r
```

Compare la version affichée avec celle d'avant ✔



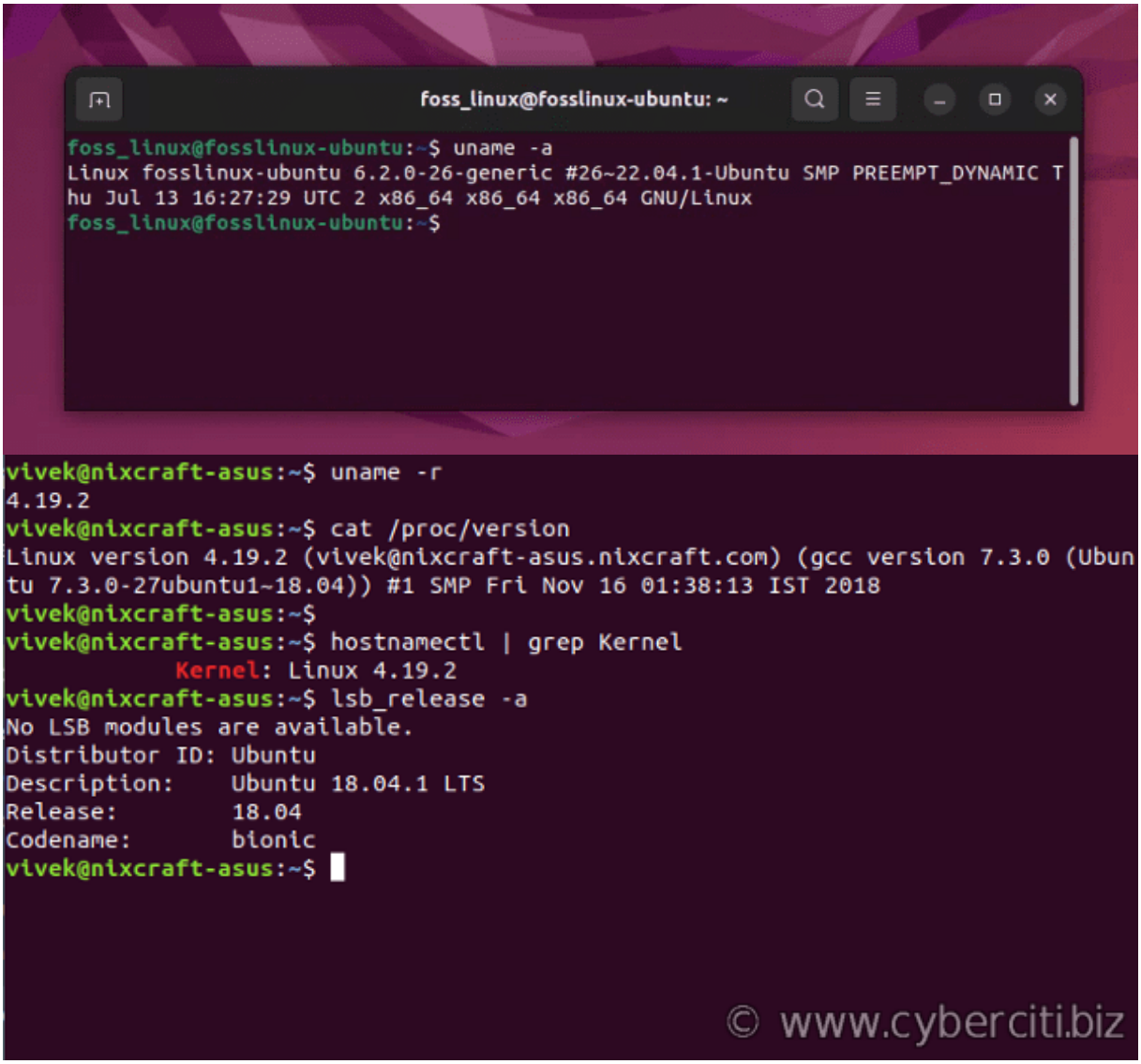
Principe général

Nous allons :

1. Télécharger le noyau depuis **kernel.org**
2. Installer les outils nécessaires
3. Compiler le noyau
4. Installer le noyau
5. Mettre à jour GRUB
6. Redémarrer et vérifier

🔎 Étape 1 : Vérifier la version actuelle du noyau

```
uname -r
```



🌐 Étape 2 : Accéder au site kernel.org

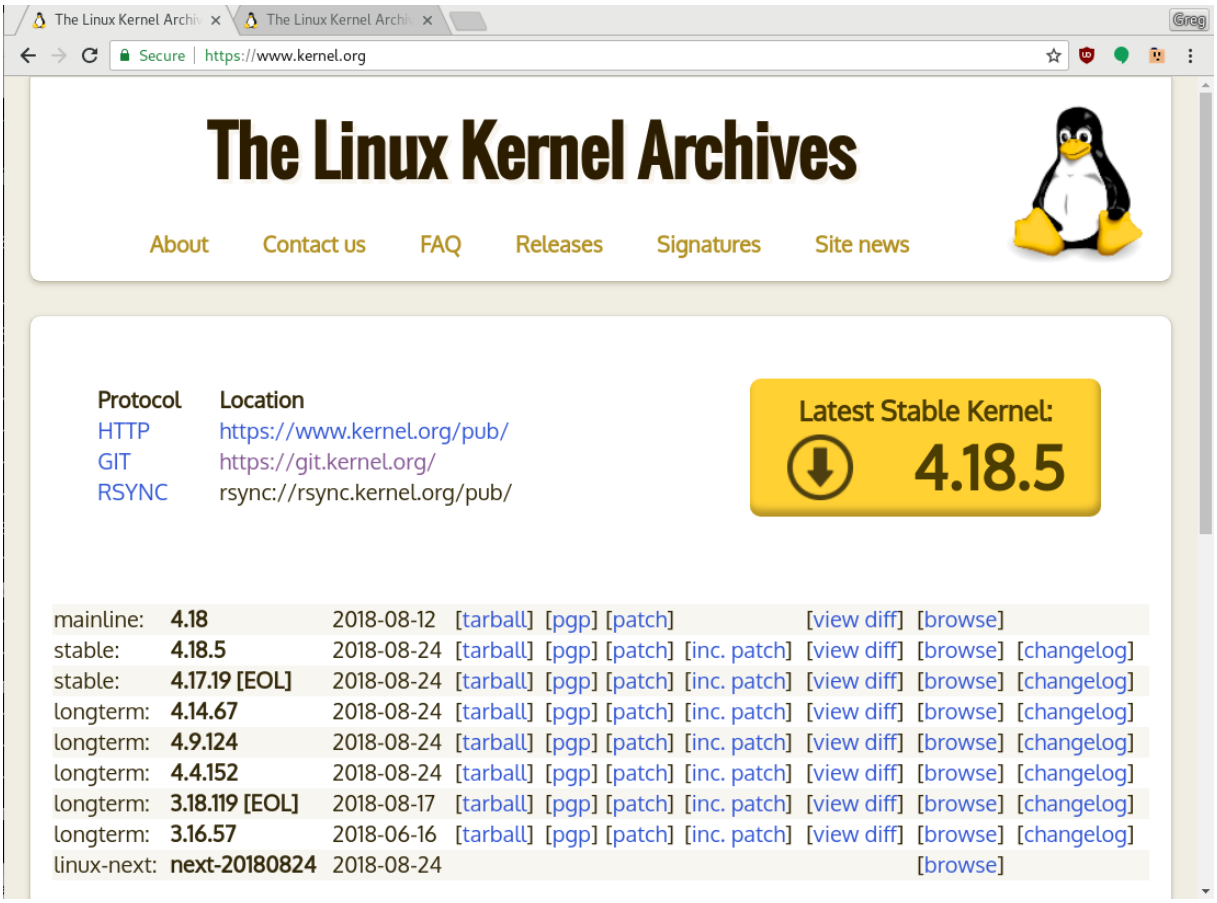
Ouvre ton navigateur et va sur :
👉 https://www.kernel.org

Tu verras plusieurs versions :

- **Stable** ✅ (recommandée)
- Longterm
- Mainline

📌 Choisis **Stable**


The Linux Kernel Archives
About
Contact us
FAQ
Releases
Signatures
Site news
Protocol
Location
HTTP
https://www.kernel.org/pub/
GIT
https://git.kernel.org/
RSYNC
rsync://rsync.kernel.org/pub/
Latest Release
6.12.1
mainline: 6.13-rc1 2024-12-01 [tarball] [patch] [view diff] [browse]
stable: 6.12.1 2024-11-22 [tarball] [pgp] [patch] [view diff] [browse] [changelog]
stable: 6.11.10 2024-11-22 [tarball] [pgp] [patch] [inc. patch] [view diff] [browse] [changelog]
longterm: 6.6.63 2024-11-22 [tarball] [pgp] [patch] [inc. patch] [view diff] [browse] [changelog]
longterm: 6.1.119 2024-11-22 [tarball] [pgp] [patch] [inc. patch] [view diff] [browse] [changelog]
longterm: 5.15.173 2024-11-17 [tarball] [pgp] [patch] [inc. patch] [view diff] [browse] [changelog]
longterm: 5.10.230 2024-11-17 [tarball] [pgp] [patch] [inc. patch] [view diff] [browse] [changelog]
longterm: 5.4.286 2024-11-17 [tarball] [pgp] [patch] [inc. patch] [view diff] [browse] [changelog]
longterm: 4.19.324 2024-11-17 [tarball] [pgp] [patch] [inc. patch] [view diff] [browse] [changelog]
linux-next: next-20241204 2024-12-04 [browse]
Other resources
Git Trees
Patchwork
Mirrors
Documentation
Wikis
Linux.com
Kernel Mailing Lists
Bugzilla
Linux Foundation
Social
Site Atom feed
Releases Atom Feed
Kernel Planet
This site is operated by the Linux Kernel Organization, Inc., a 501(c)3 nonprofit corporation, with support from the following sponsors.
EQUINIX
METAL
fastly
CONSTELLIX
Sponsored by Red Hat
PURESTORAGE
Google
THE LINUX FOUNDATION



📥 Étape 3 : Télécharger le noyau Linux (archive .tar.xz)

Clique sur la version stable, par exemple :

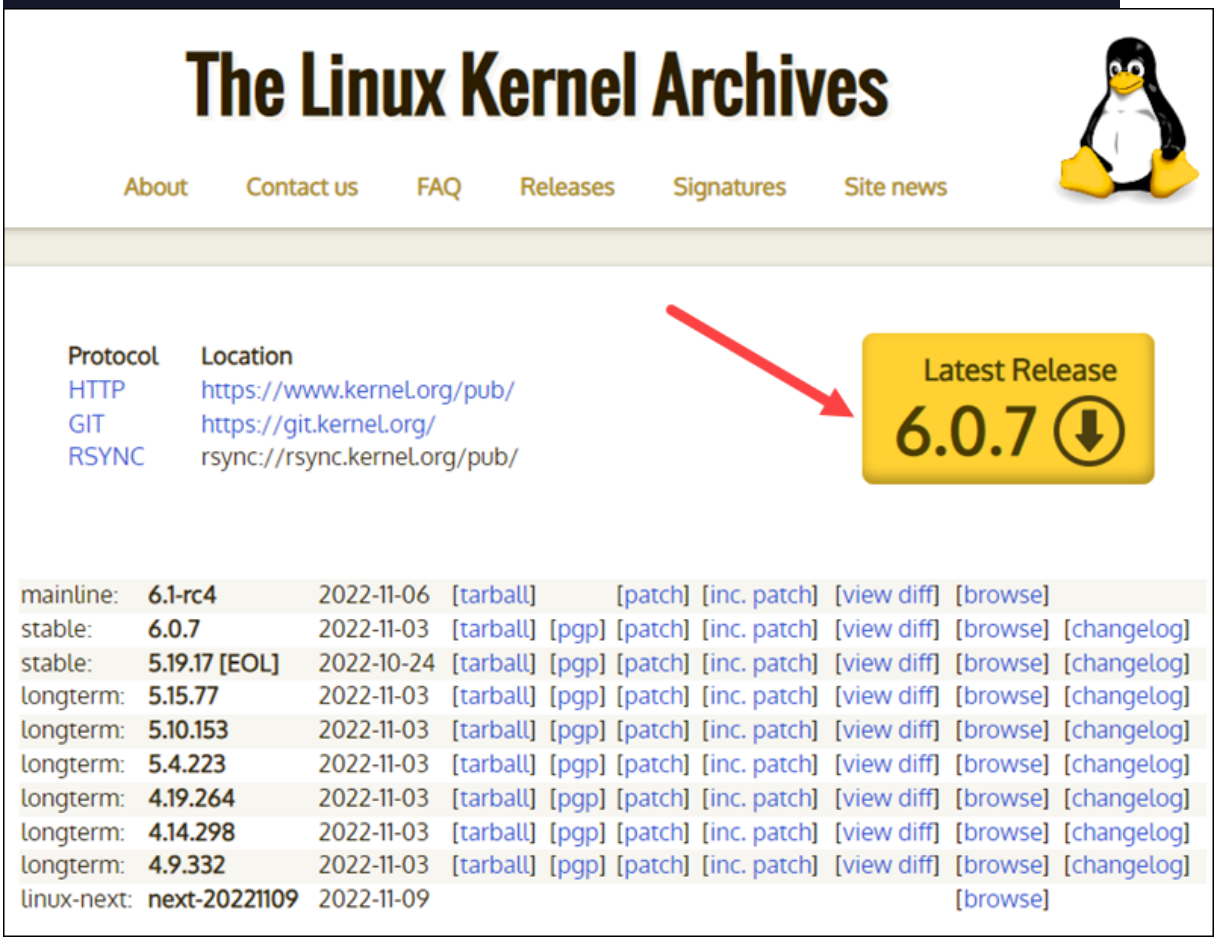
```
linux-6.x.y.tar.xz
```

Ou via terminal :

```
cd ~/Téléchargements
wget
https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-
6.x.y.tar.xz
```

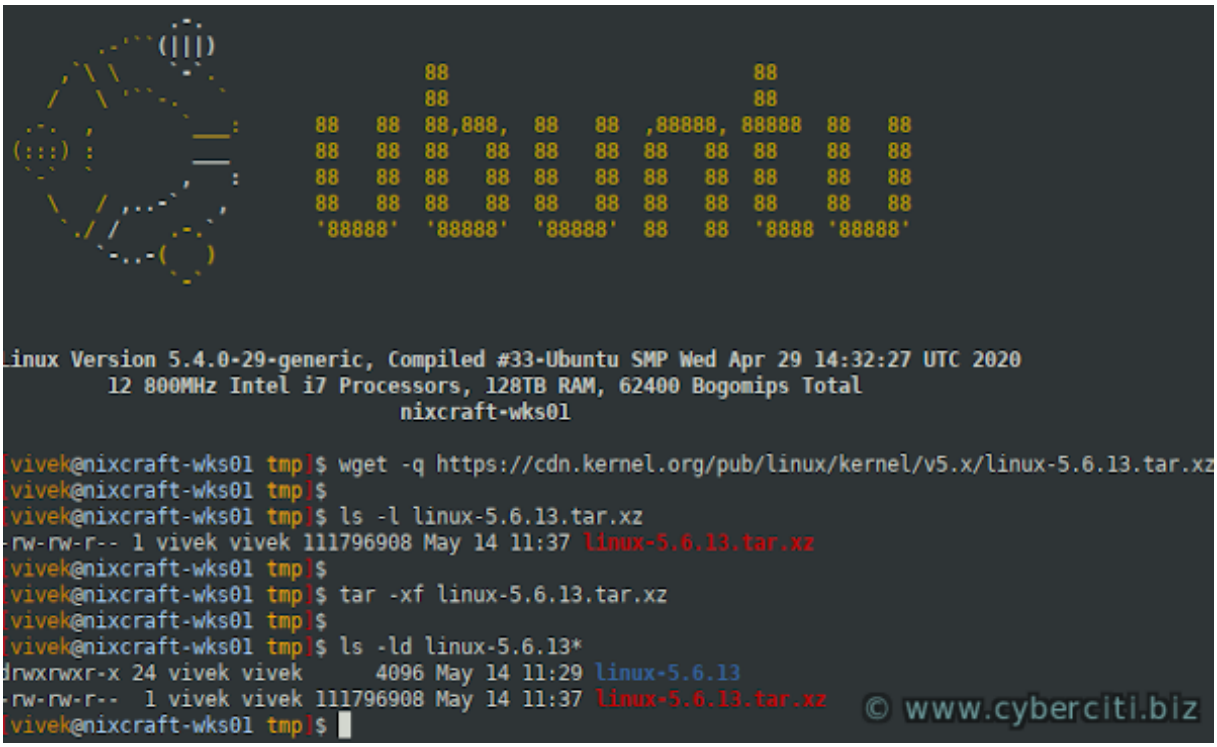
(remplace `6.x.y` par la version exacte)

```
root@voxfor:~# tar -xvf file.tar.xz
hello_world.pl
snap/
snap/lxd/
snap/lxd/common/
snap/lxd/27037/
snap/lxd/current
root@voxfor:~#
```



📦 Étape 4 : Extraire l’archive du noyau

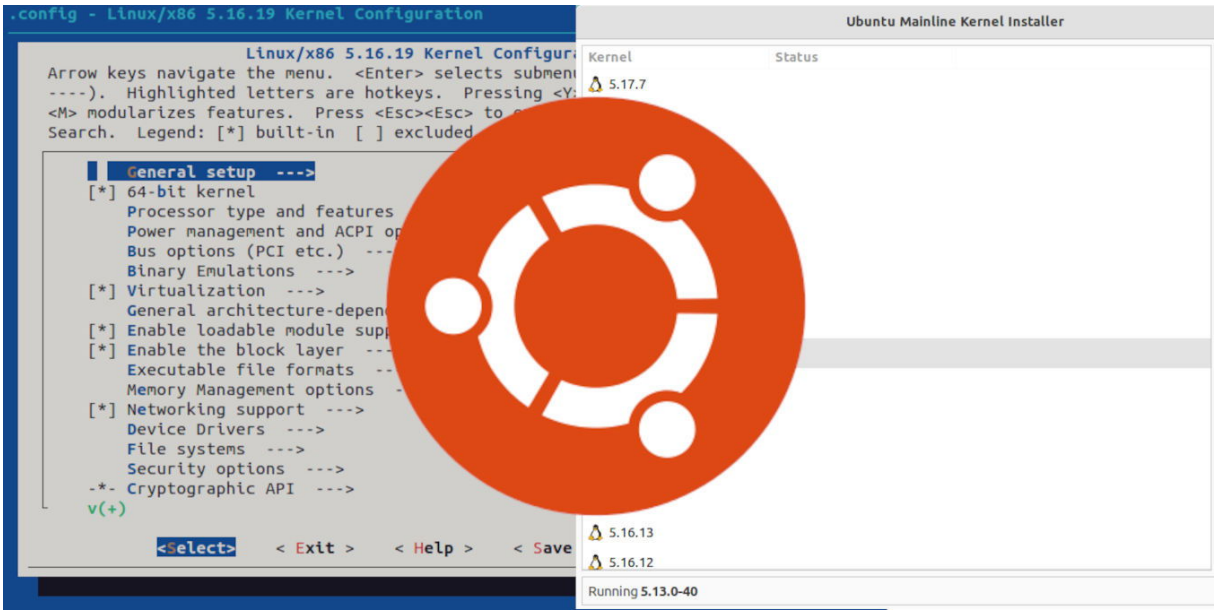
```
tar -xvf linux-6.x.y.tar.xz
cd linux-6.x.y
```


88 88
88 88
88 88 88,888, 88 88 ,88888, 88888 88 88
88 88 88 88 88 88 88 88 88 88 88 88
88 88 88 88 88 88 88 88 88 88 88 88
'88888' '88888' '88888' 88 88 '8888 '88888'
Linux Version 5.4.0-29-generic, Compiled #33-Ubuntu SMP Wed Apr 29 14:32:27 UTC 2020
12 800MHz Intel i7 Processors, 128TB RAM, 62400 Bogomips Total
nixcraft-wks01
[vivek@nixcraft-wks01 tmp]$ wget -q https://cdn.kernel.org/pub/linux/kernel/v5.x/linux-5.6.13.tar.xz
[vivek@nixcraft-wks01 tmp]$
[vivek@nixcraft-wks01 tmp]$ ls -l linux-5.6.13.tar.xz
-rw-rw-r-- 1 vivek vivek 111796908 May 14 11:37 linux-5.6.13.tar.xz
[vivek@nixcraft-wks01 tmp]$
[vivek@nixcraft-wks01 tmp]$ tar -xf linux-5.6.13.tar.xz
[vivek@nixcraft-wks01 tmp]$
[vivek@nixcraft-wks01 tmp]$ ls -ld linux-5.6.13*
drwxrwxr-x 24 vivek vivek 4096 May 14 11:29 linux-5.6.13
-rw-rw-r-- 1 vivek vivek 111796908 May 14 11:37 linux-5.6.13.tar.xz
[vivek@nixcraft-wks01 tmp]$
© www.cyberciti.biz



🧰 Étape 5 : Installer les dépendances nécessaires

```
sudo apt update
sudo apt install build-essential libncurses-dev bison
flex libssl-dev libelf-dev
```





🔧 Pourquoi installer les dépendances avant de compiler le noyau Linux ?

Quand tu télécharges le noyau depuis **kernel.org**, tu récupères **le code source brut**.
👉 Pour transformer ce code source en **noyau exécutable**, Ubuntu a besoin **d'outils et de bibliothèques spécifiques**.

Sans ces dépendances :
❌ la compilation échoue

❌ certaines fonctionnalités du noyau sont absentes
❌ erreurs du type *“command not found”* ou *“missing library”*

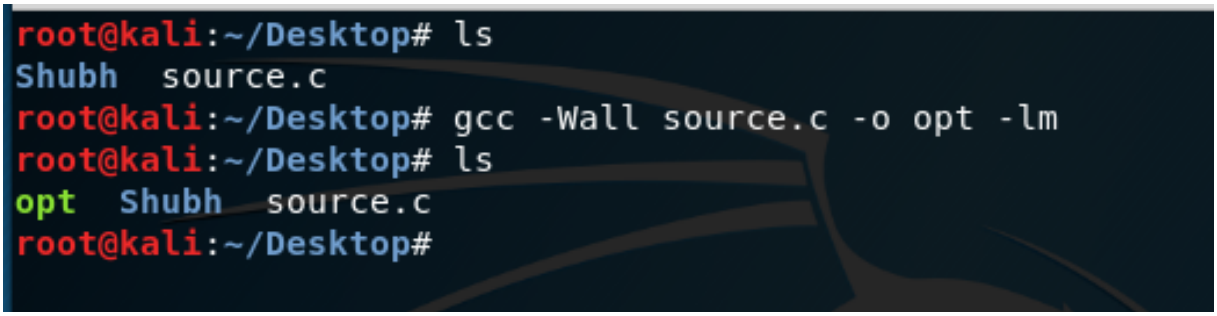
📦 Dépendances installées à l’étape 5

Commande utilisée :

```
sudo apt install build-essential libncurses-dev bison flex libssl-dev libelf-dev
```

Nous allons détailler **chaque paquet**, son rôle, et **ce qui se passe s’il manque**.

🛠️ 1. `build-essential` — outils de compilation





Contient :

- `gcc` → compilateur C
- `g++` → compilateur C++
- `make` → exécute le Makefile
- bibliothèques standard (`libc`)

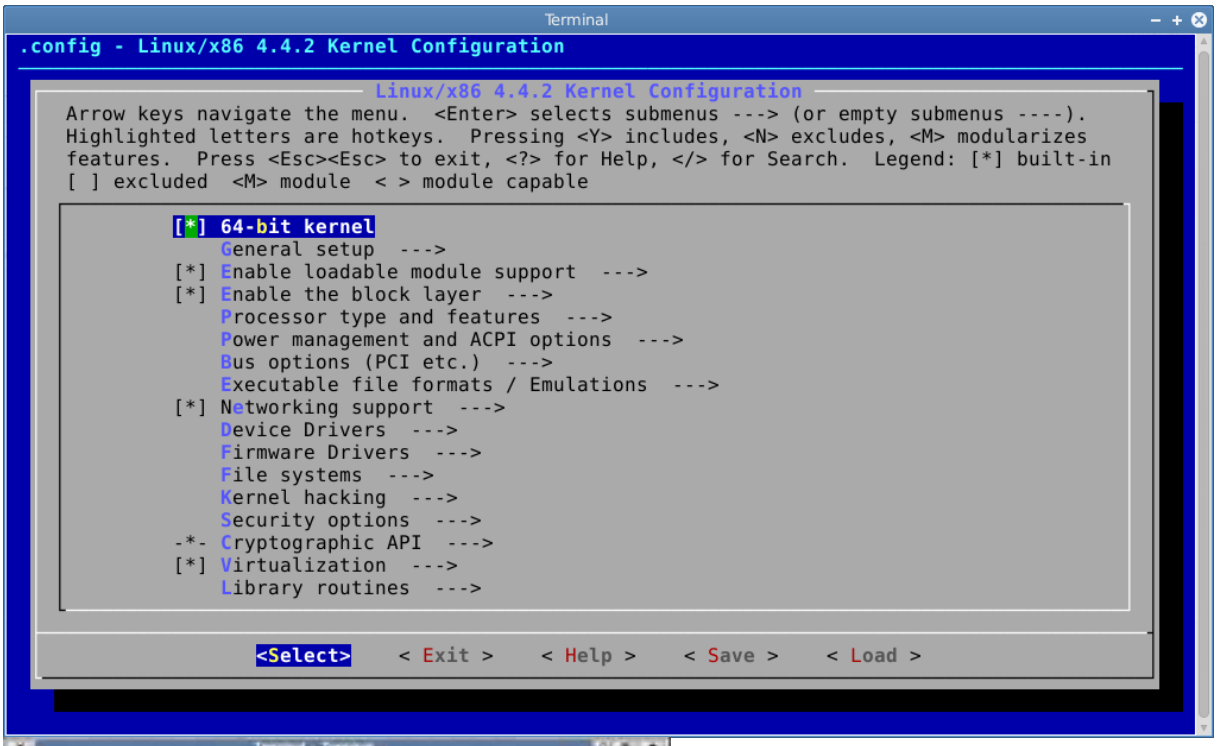
Utilité :

✓ Compiler le code source du noyau
✓ Interpréter les fichiers `Makefile`

✗ Sans lui :

```
make: command not found
gcc: command not found
```

🧩 2. `libncurses-dev` — interface de configuration





Utilité :

- Interface **texte interactive**
- Utilisée par :

```
make menuconfig
make oldconfig
```

✓ Permet de sélectionner les fonctionnalités du noyau
❌ Sans lui :

```
menuconfig: No curses library found
```

🧠 3. `bison` — analyse syntaxique

```
linux@ubuntu:~/files$ bison -p FILE file.y
linux@ubuntu:~/files$ grep FILE file.tab.c
#define yyparse         FILEparse
#define yylex           FILElex
#define yyerror         FILEerror
#define yydebug         FILEdebug
#define yynerrs         FILEnerrs
#define yylval          FILElval
#define yychar          FILEchar
extern int FILEdebug;
extern YYSTYPE FILElval;
int FILEparse (void);
yy_symbol_value_print (FILE *yyoutput, int yytype, YYSTYPE const * const yyvaluep)
  FILE *yyo = yyoutput;
yy_symbol_print (FILE *yyoutput, int yytype, YYSTYPE const * const yyvaluep)
linux@ubuntu:~/files$
```

Utilité :

- Génère des **analyseurs syntaxiques**
- Utilisé pour traiter certaines parties du noyau

❌ Sans lui :

```
error: bison not found
```

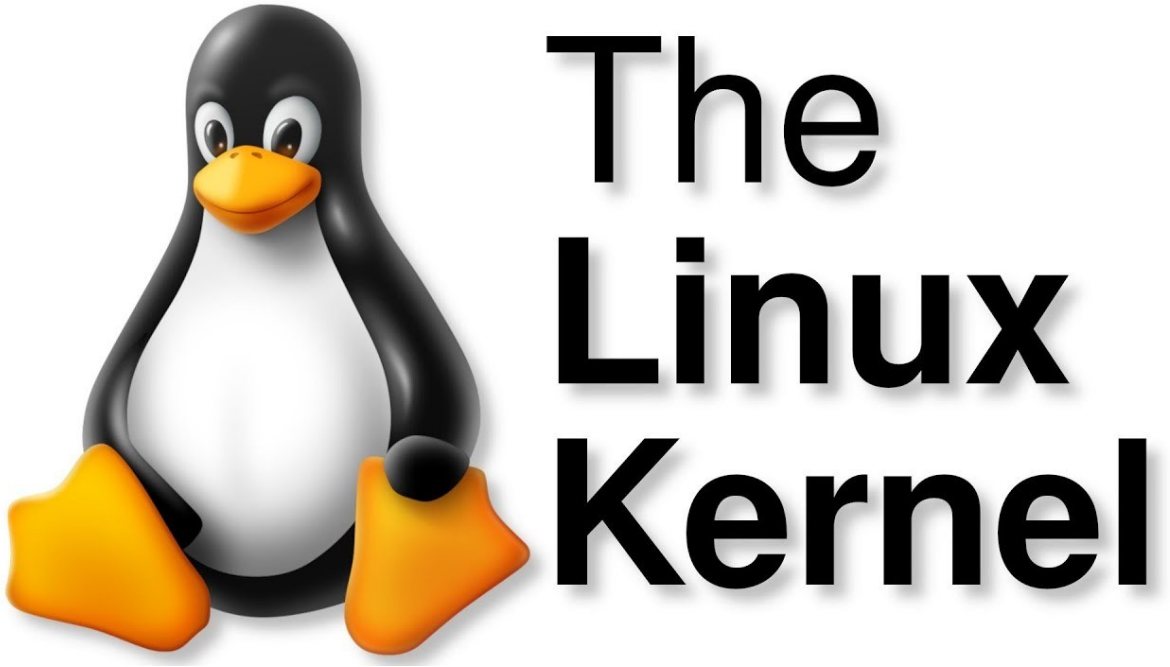
🧬 4. `flex` — analyse lexicale

```
marius@softpedia:~/Desktop/flex-2.5.39
File  Edit  View  Search  Terminal  Help
[marius@softpedia flex-2.5.39]$ ./flex --help
Usage: flex [OPTIONS] [FILE]...
Generates programs that perform pattern-matching on text.

Table Compression:
  -Ca, --align      trade off larger tables for better memory alignment
  -Ce, --ecs        construct equivalence classes
  -Cf               do not compress tables; use -f representation
  -CF               do not compress tables; use -F representation
  -Cm, --meta-ecs   construct meta-equivalence classes
  -Cr, --read       use read() instead of stdio for scanner input
  -f, --full        generate fast, large scanner. Same as -Cfr
  -F, --fast        use alternate table representation. Same as -CFr
  -Cem              default compression (same as --ecs --meta-ecs)

Debugging:
  -d, --debug             enable debug mode in scanner
  -b, --backup            write backing-up information to lex.backup
  -p, --perf-report       write performance report to stderr
  -s, --nodefault         suppress default rule to ECHO unmatched text
  -T, --trace             flex should run in trace mode
  -w, --nowarn            do not generate warnings
  -v, --verbose           write summary of scanner statistics to stdout

Files:
  -o, --outfile=FILE      specify output filename
  -S, --skel=FILE         specify skeleton file
  -t, --stdout            write scanner on stdout instead of lex.yy.c
```



Utilité :

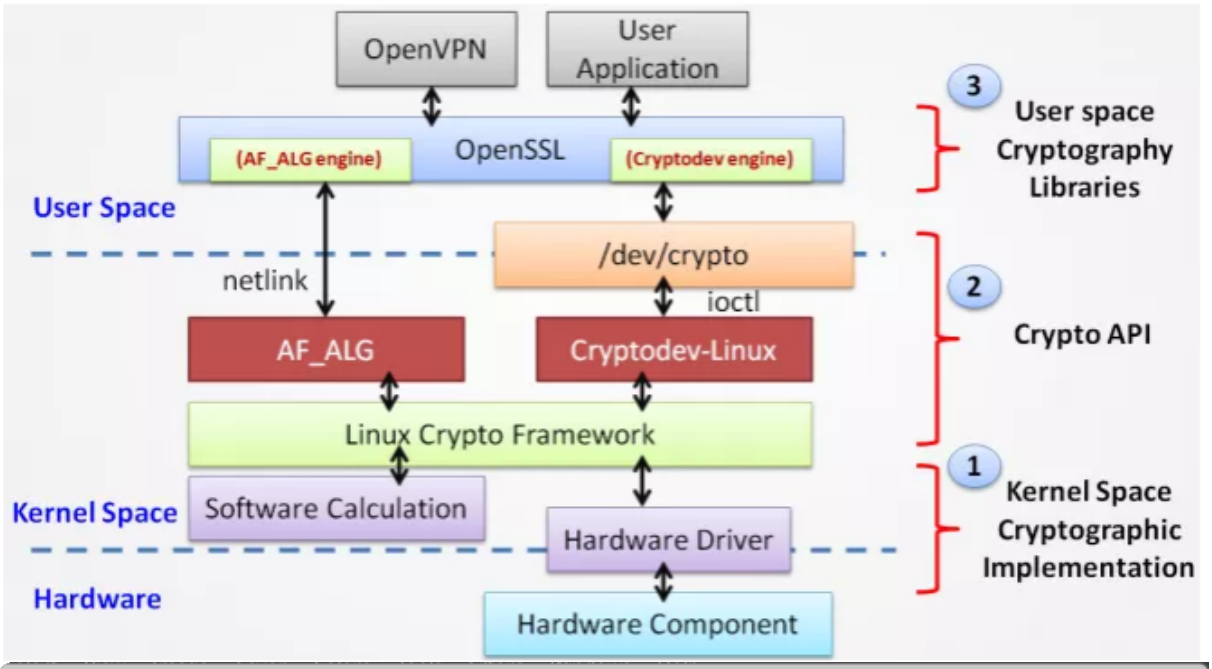
- Complète `bison`
- Analyse les **symboles et mots-clés** du code source

❌ Sans lui :

```
flex: command not found
```

---

5. `libssl-dev` — sécurité et cryptographie




```
File  Edit  View  Search  Terminal  Help
Troller$ modinfo hid
filename:       /lib/modules/[redacted]-generic/kernel/drivers/hid/hid.ko.zst
license:        GPL
description:    HID support for Linux
author:         Jiri Kosina
author:         Vojtech Pavlik
author:         Andreas Gal
srcversion:     [redacted]
depends:
intree:         Y
name:           hid
vermagic:       [redacted]-generic SMP preempt mod_unload modversions [redacted]
sig_id:         PKCS#7
signer:         Build time autogenerated kernel key
sig_key:        [redacted]08:4A[redacted]
sig_hashalgo:   sha512
signature:      [redacted]82:4A[redacted]
```

Utilité :

- Gestion du chiffrement
- Signature des modules du noyau
- Sécurité réseau

❌ Sans lui :

```
OpenSSL headers missing
```

---

6. `libelf-dev` — format ELF



Utilité :

- Manipulation des fichiers ELF
- Nécessaire pour :
  - modules du noyau
  - symboles de débogage

❌ Sans lui :

```
libelf not found
```

📊 Résumé clair

Dépendance	Rôle principal
build-essential	Compilation (gcc, make)
libncurses-dev	Interface de configuration
bison	Analyse syntaxique
flex	Analyse lexicale
libssl-dev	Sécurité / cryptographie
libelf-dev	Gestion des modules ELF

⚙️ Étape 6 : Copier la configuration actuelle (recommandé)

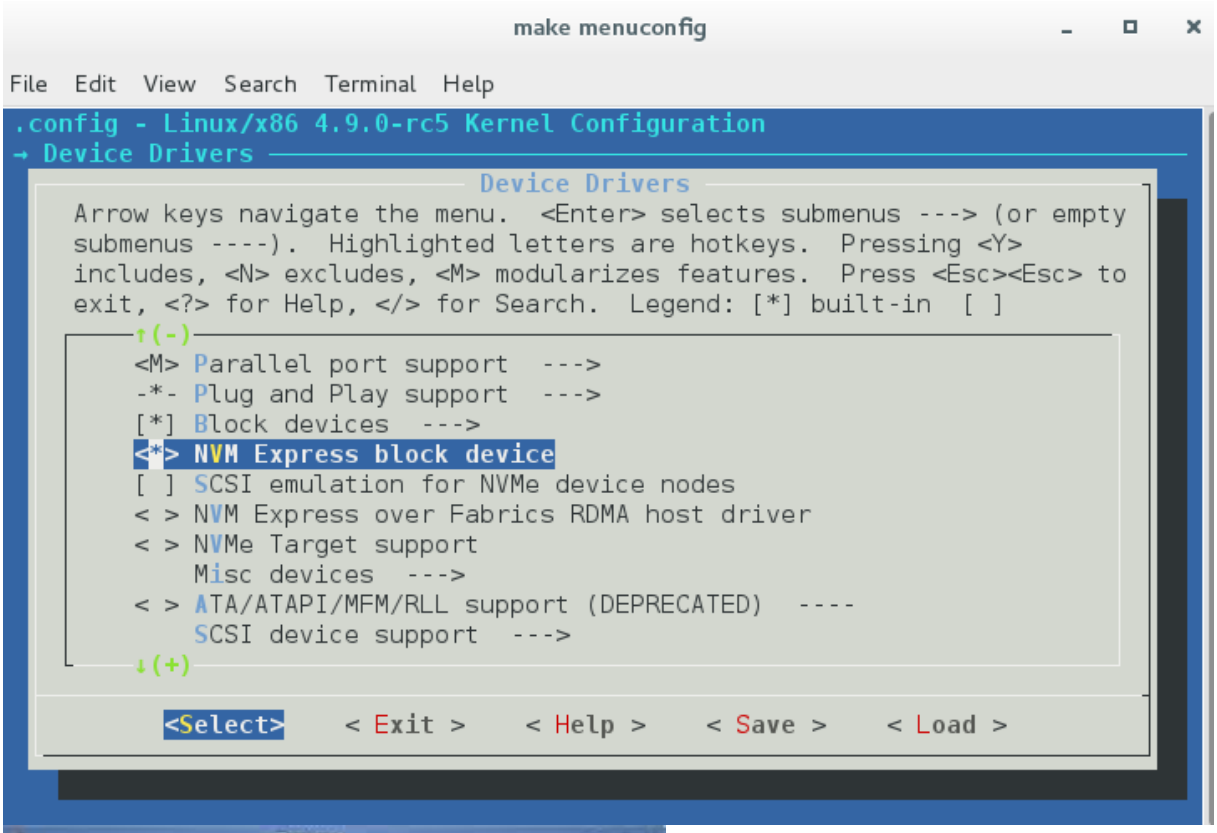
Cela évite beaucoup d'erreurs.

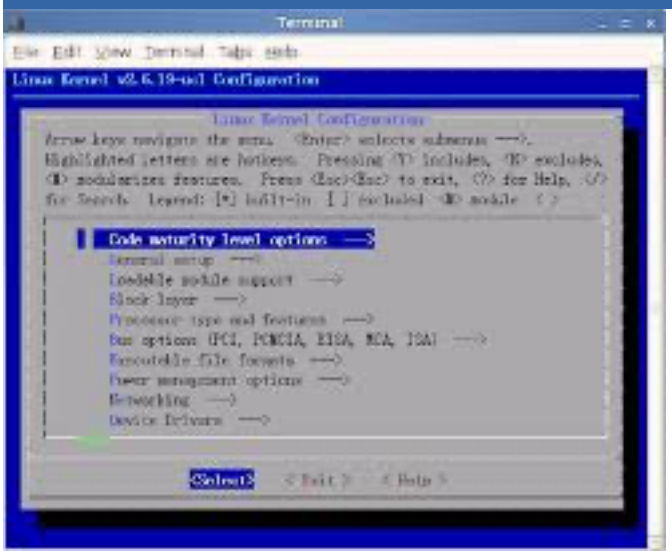
```
cp /boot/config-$(uname -r) .config
```

Puis mettre à jour la configuration :

```
make oldconfig
```

(Appuie sur **Entrée** pour accepter les valeurs par défaut)

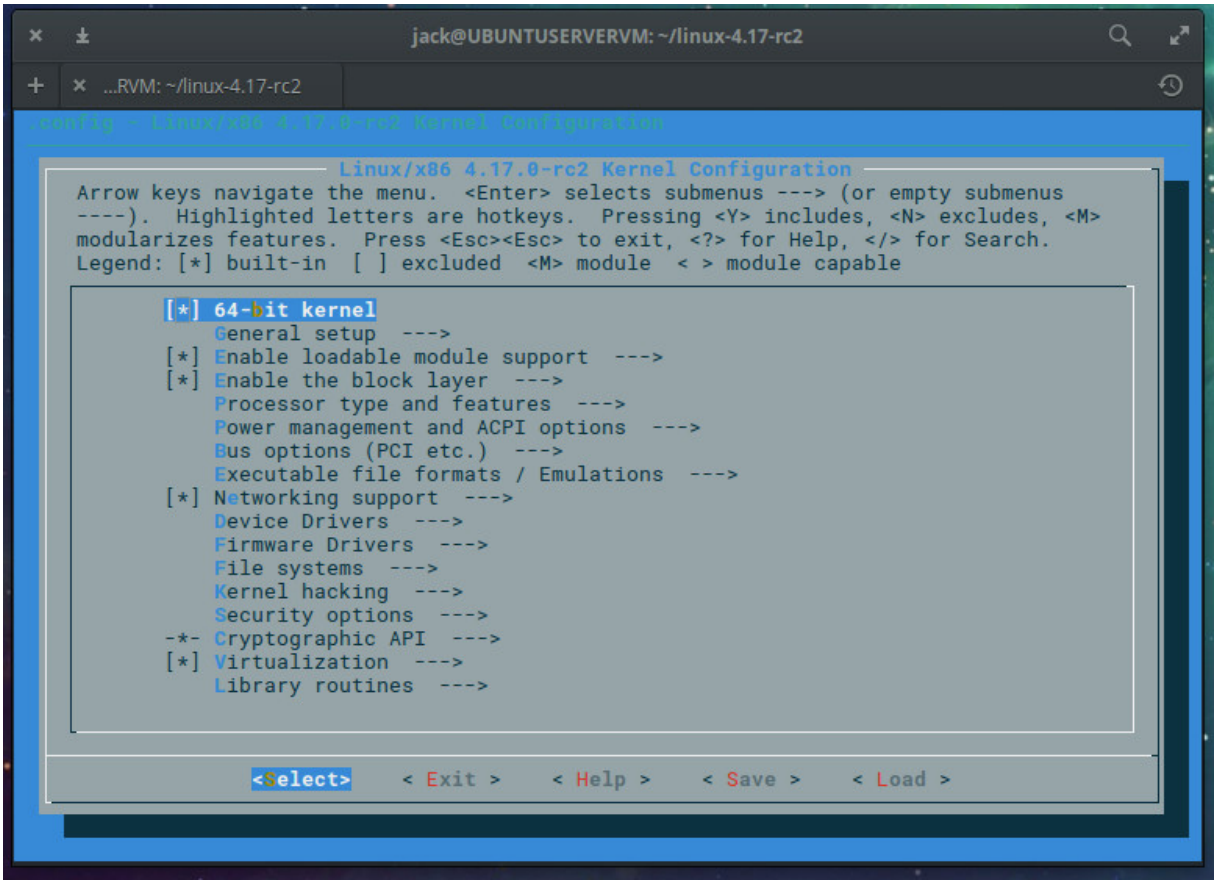


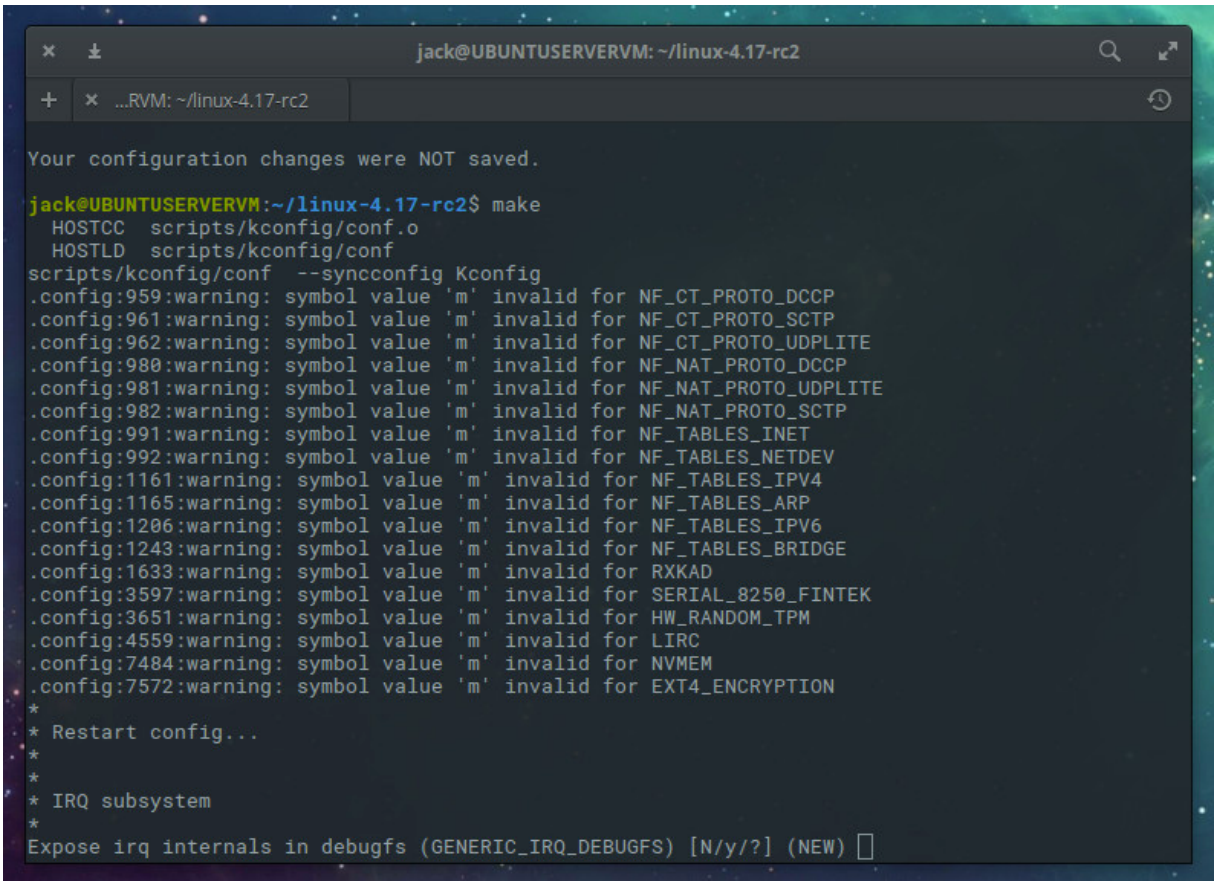


🛠️ Étape 7 : Compiler le noyau (⏳ long)

⚠️ Peut prendre **30 à 90 minutes** selon ta machine virtuelle.

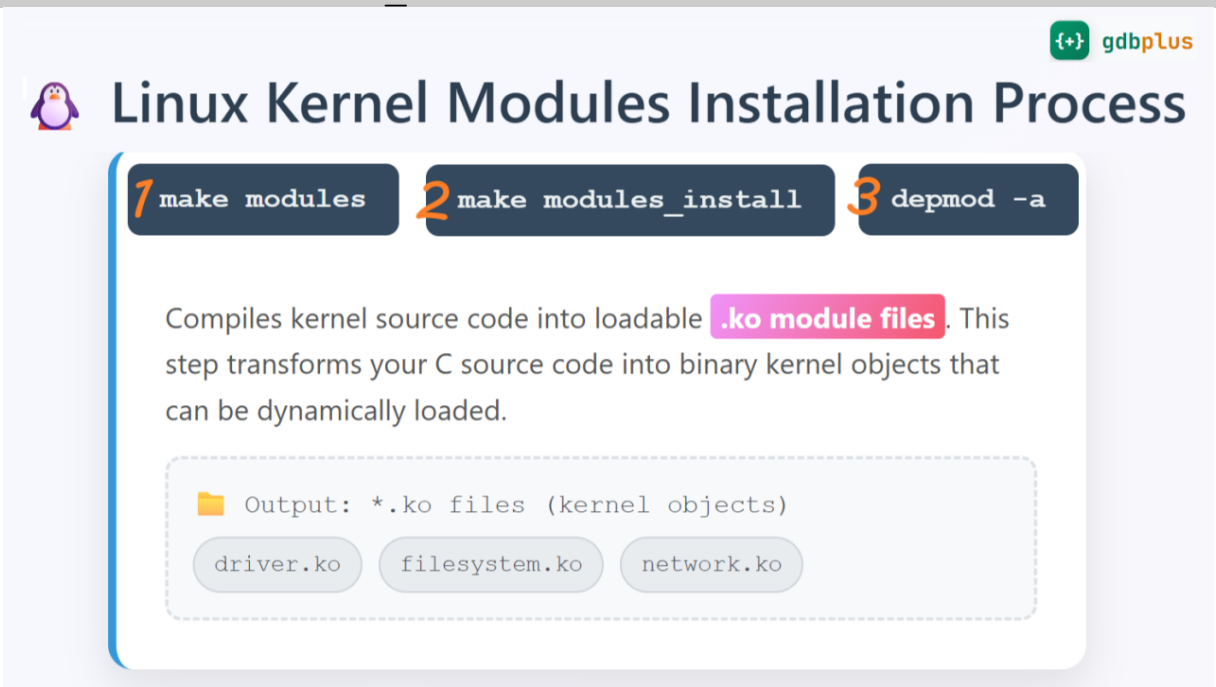
```
make -j$(nproc)
```

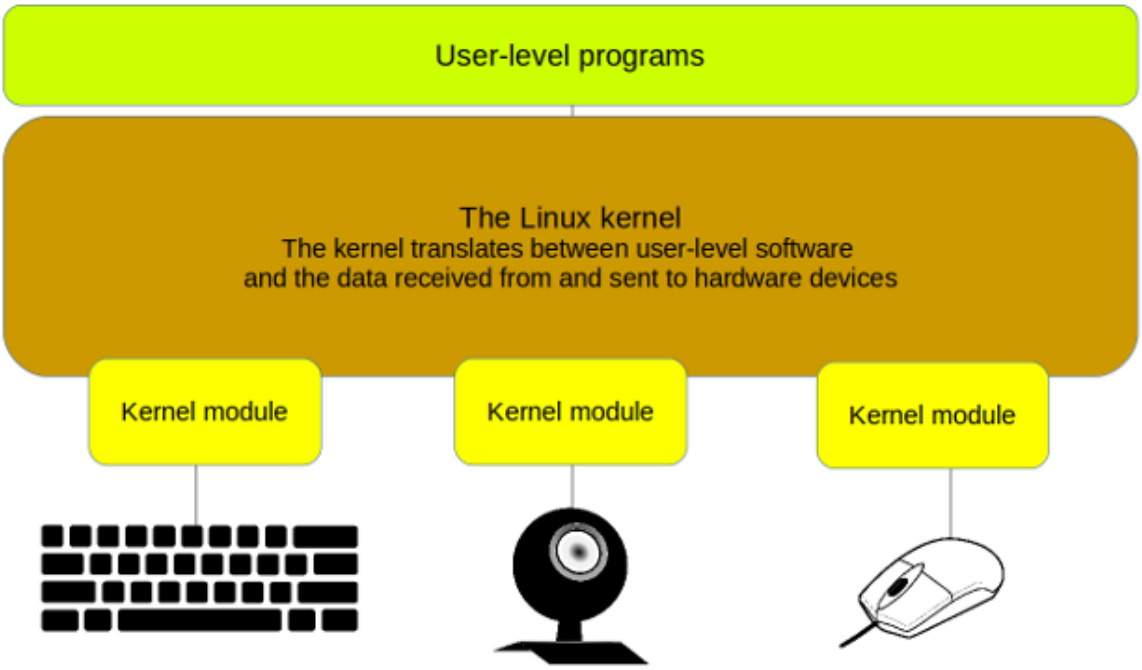




📥 Étape 8 : Installer les modules du noyau

```
sudo make modules_install
```





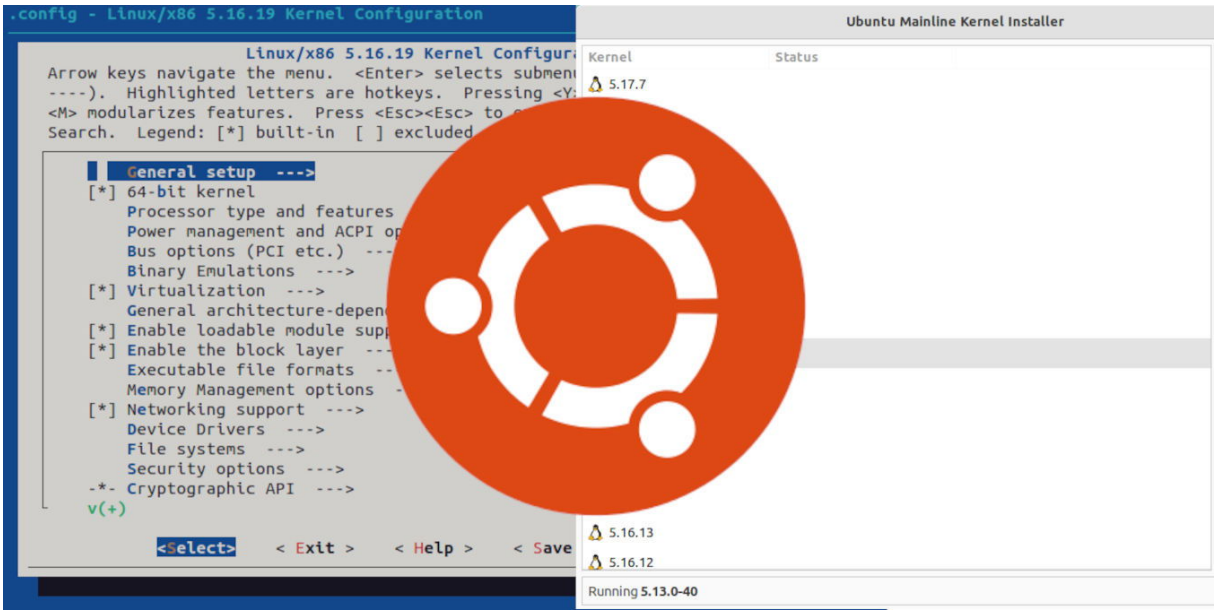
🧩 Étape 9 : Installer le noyau

```
sudo make install
```

📌 Cela installe :

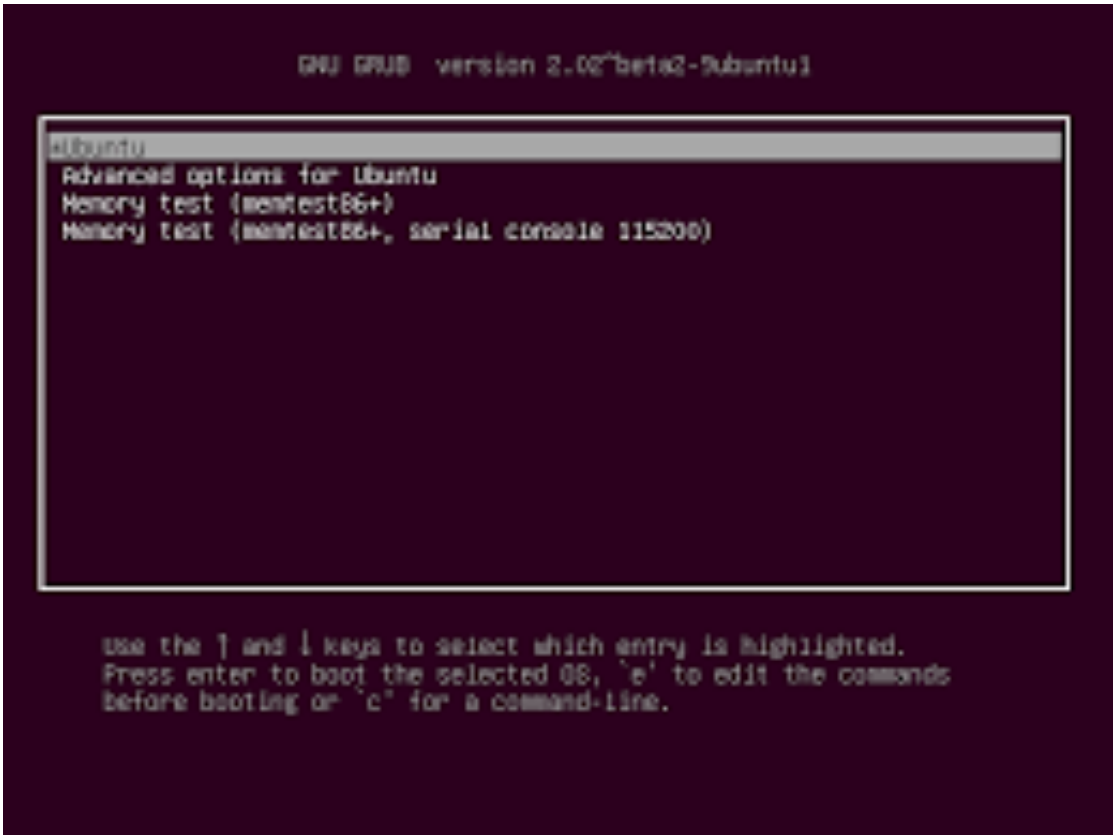
- le noyau dans `/boot`
- met à jour GRUB automatiquement

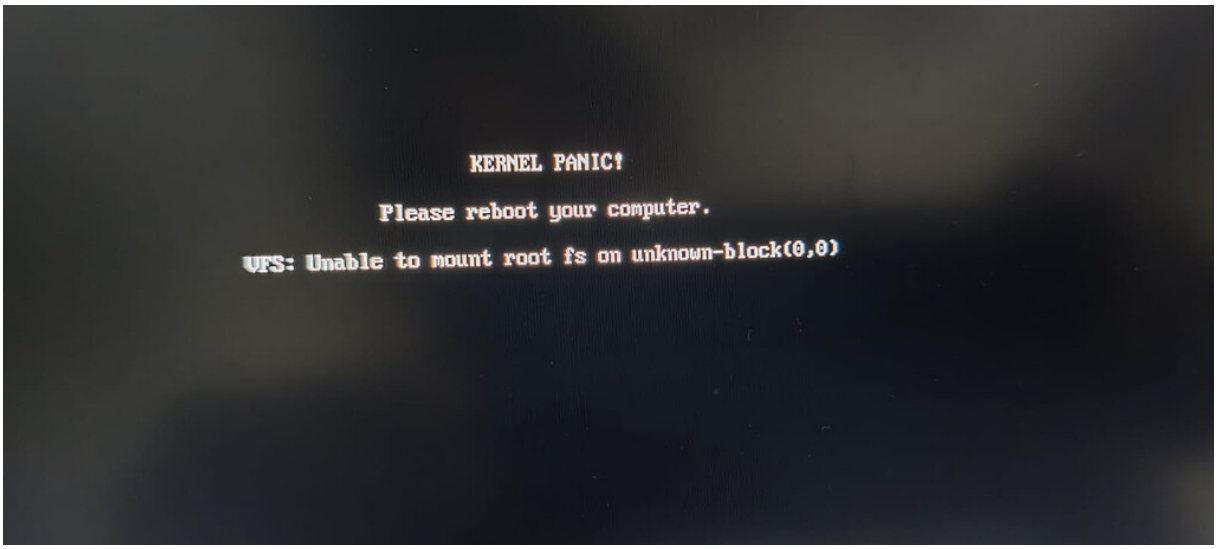
```
marko@pnap:~/linux-6.0.7$ sudo make install
sh ./arch/x86/boot/install.sh 6.0.7 arch/x86/boot/bzImage \
        System.map "/boot"
run-parts: executing /etc/kernel/postinst.d/apt-auto-removal 6.0.7 /boot/vmlinuz-6.0.7
run-parts: executing /etc/kernel/postinst.d/dkms 6.0.7 /boot/vmlinuz-6.0.7
 * dkms: running auto installation service for kernel 6.0.7                  [ OK ]
run-parts: executing /etc/kernel/postinst.d/initramfs-tools 6.0.7 /boot/vmlinuz-6.0.7
update-initramfs: Generating /boot/initrd.img-6.0.7
run-parts: executing /etc/kernel/postinst.d/update-notifier 6.0.7 /boot/vmlinuz-6.0.7
run-parts: executing /etc/kernel/postinst.d/zz-update-grub 6.0.7 /boot/vmlinuz-6.0.7
Sourcing file `/etc/default/grub'
Sourcing file `/etc/default/grub.d/init-select.cfg'
Generating grub configuration file ...
done
marko@pnap:~/linux-6.0.7$
```



Étape 10 : Redémarrer la machine

```
sudo reboot
```



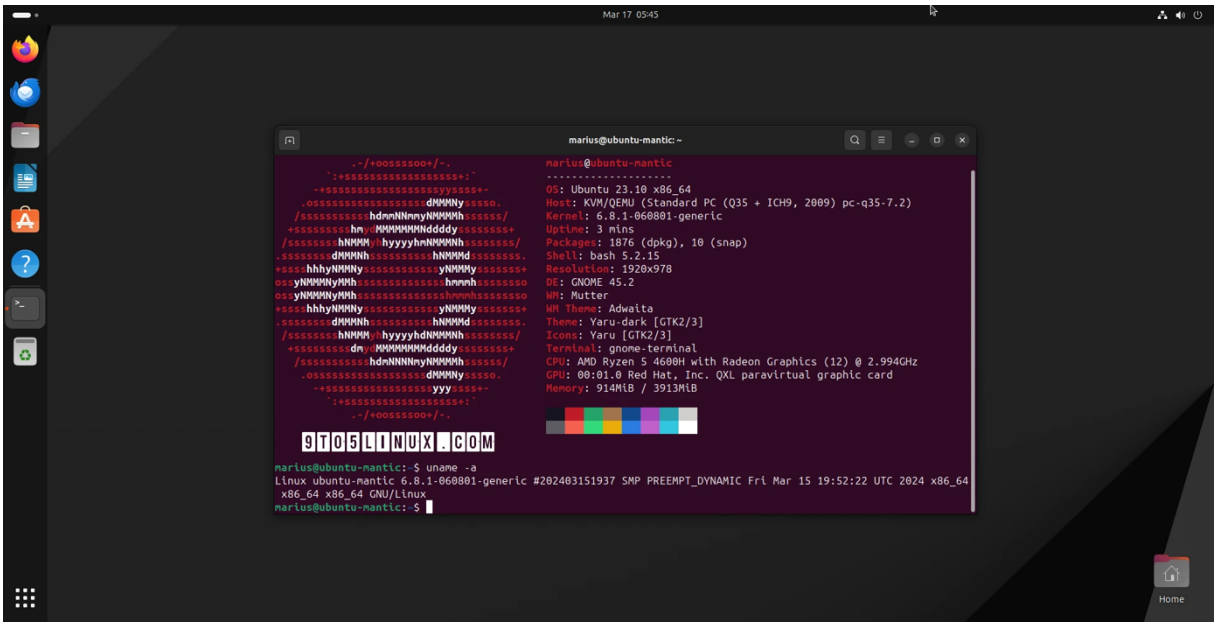


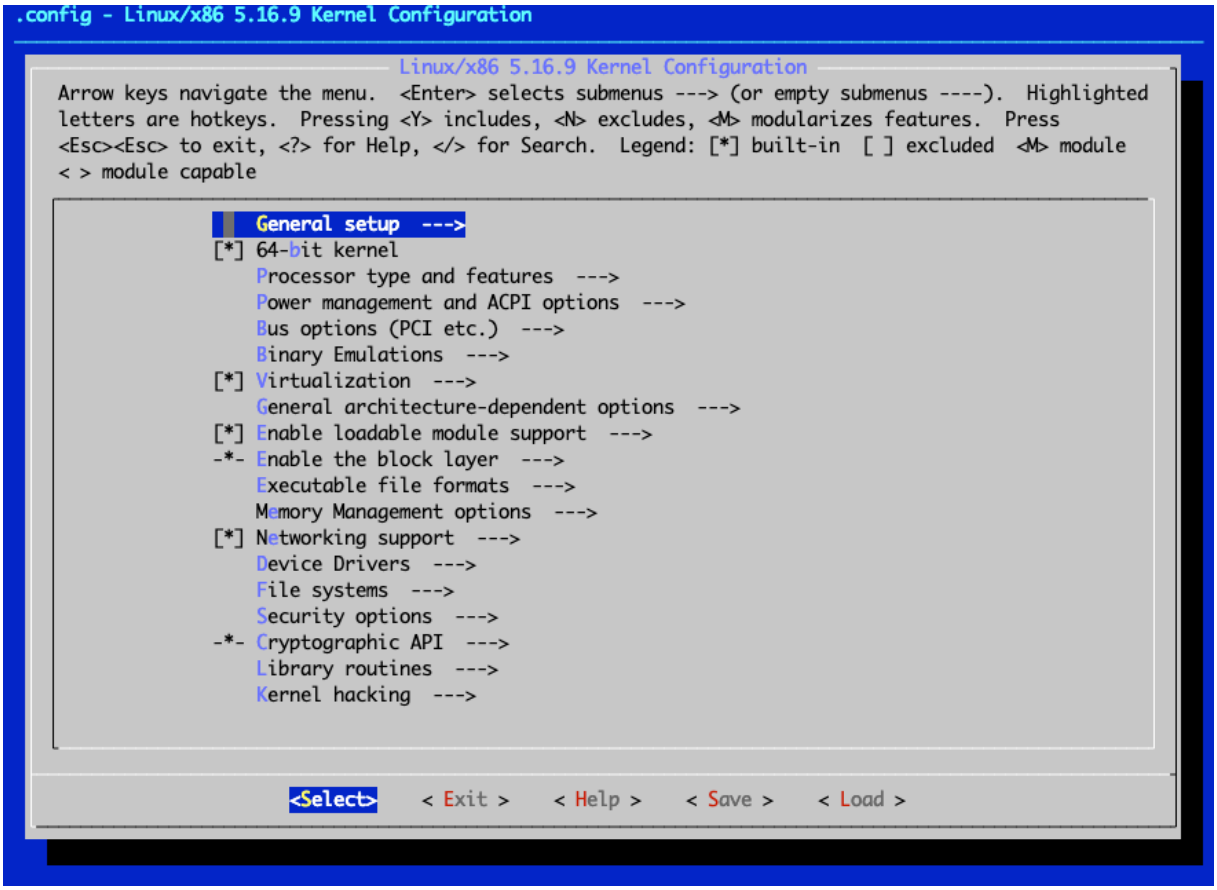
🧪 Étape 11 : Vérifier le nouveau noyau

Après redémarrage :

```
uname -r
```

✅ La version doit correspondre au noyau téléchargé.

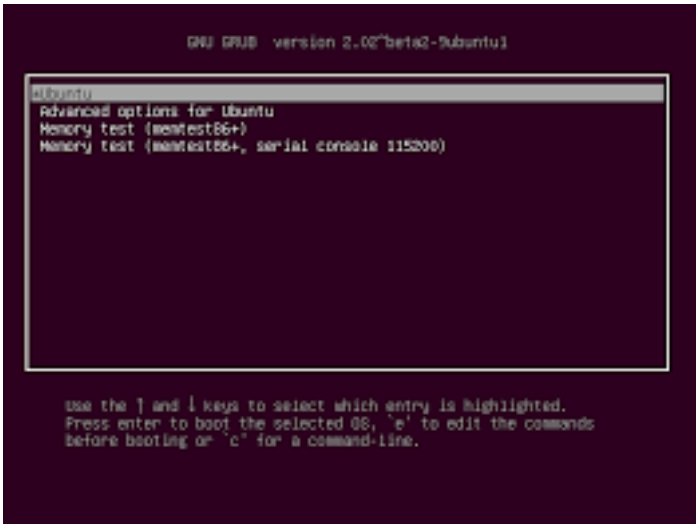


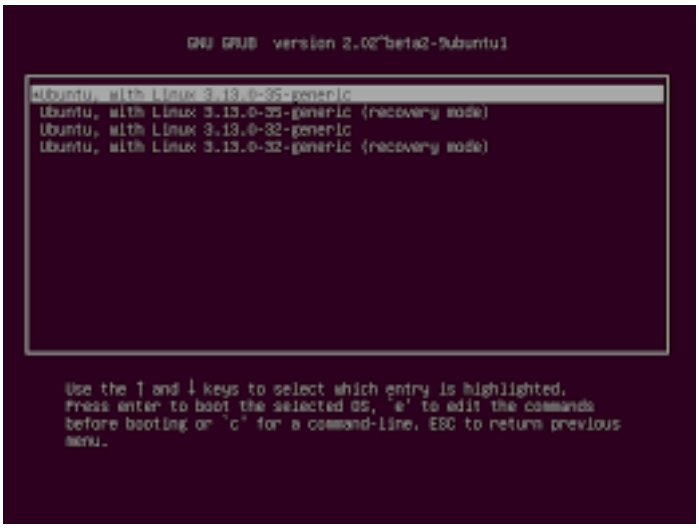


En cas de problème (très important)

Au démarrage :

1. Appuie sur **Shift** ou **Esc**
2. **Options avancées pour Ubuntu**
3. Choisis un **ancien noyau**





📌 Résumé

Étape	Action
1	Télécharger le noyau
2	Extraire l'archive
3	Installer dépendances
4	Configurer
5	Compiler
6	Installer
7	Redémarrer
8	Vérifier

🔧 Gestion des modules du noyau Linux

🧠 Qu'est-ce qu'un module du noyau ?

Un **module du noyau (kernel module)** est un morceau de code **chargé dynamiquement** dans le noyau Linux **sans redémarrer** le système.

📌 Exemples :

- Pilotes de périphériques (USB, carte réseau, Wi-Fi)
- Systèmes de fichiers (ext4, vfat)
- Fonctions réseau ou sécurité

👉 Les modules ont l'extension `.ko` (*kernel object*).

🎯 Pourquoi utiliser des modules ?

✔ Éviter un noyau trop volumineux
✔ Charger uniquement ce qui est nécessaire
✔ Ajouter / retirer des fonctionnalités à chaud
✔ Faciliter la maintenance et le débogage

📁 Où sont stockés les modules ?

```
/lib/modules/$(uname -r)/
```

Structure typique :

```
kernel/
 ├── drivers/
 ├── fs/
 ├── net/
 └── sound/
```

🛠️ Commandes essentielles de gestion des modules

🔍 1. Lister les modules chargés

```
lsmod
```

Affiche :

- nom du module
- taille
- nombre d'utilisations

📥 2. Charger un module

```
sudo modprobe nom_module
```

Exemple :

```
sudo modprobe usb_storage
```

✔ Charge automatiquement les dépendances

📤 3. Décharger un module

```
sudo modprobe -r nom_module
```

ou

```
sudo rmmod nom_module
```

⚠️ Impossible si le module est en cours d'utilisation

ℹ 4. Obtenir des informations sur un module

```
modinfo nom_module
```

Affiche :

- description
- auteur
- licence
- dépendances

🔎 5. Vérifier les messages du noyau

```
dmesg | tail
```

Utile après :

- chargement
- erreur de module
- périphérique branché

⚙️ Chargement automatique des modules au démarrage

📄 Méthode 1 : fichier `/etc/modules`

```
sudo nano /etc/modules
```

Ajouter :

```
nom_module
```

📄 Méthode 2 : configuration modprobe

```
/etc/modprobe.d/mon_module.conf
```

Exemple :

```
options snd-hda-intel model=auto
```

🚫 Blacklist (interdire un module)

Empêcher le chargement d'un module :

```
sudo nano /etc/modprobe.d/blacklist.conf
```

Ajouter :

```
blacklist nom_module
```

📌 Utile pour :

- pilotes problématiques
- conflits matériels

🧩 Modules et compilation du noyau

Lors de **`make menuconfig`** :

- `[*]` → intégré directement au noyau
- `[M]` → compilé comme **module**
- `[ ]` → désactivé

📌 Les modules sont ensuite installés avec :

```
sudo make modules_install
```

📌 Résumé synthétique

Action	Commande
Lister modules	`lsmod`
Charger	`modprobe`
Décharger	`modprobe -r`
Infos	`modinfo`

Messages noyau	dmesg

🎓 Résumé

La gestion des modules du noyau Linux permet d'ajouter ou de retirer dynamiquement des fonctionnalités sans redémarrage du système, offrant ainsi flexibilité, performance et facilité de maintenance.

🔧 Exemple réel : gestion du module `usb_storage`

📌 Pourquoi ce module ?

- Présent par défaut sur Ubuntu
- Facile à tester
- Utilisé pour les **clés USB**
- Sans risque pour le système

🔍 Étape 1 : Vérifier si le module est chargé

```
lsmod | grep usb_storage
```

🔎 Résultat possible :

```
usb_storage        73728  0
```

👉 Cela signifie :

- `usb_storage` est chargé
- Taille : 73728 octets
- Utilisé par 0 module

📤 Étape 2 : Décharger le module

```
sudo modprobe -r usb_storage
```

ou :

```
sudo rmmod usb_storage
```

🔎 **Vérification :**

```
lsmod | grep usb_storage
```

✅ **Aucune sortie** → module déchargé

🧪 Effet réel observé

👉 Branche une **clé USB** :

- ❌ Elle **n'apparaît pas**
- ❌ Pas montée automatiquement

📌 Cela prouve que le module est bien désactivé.

📥 Étape 3 : Recharger le module

```
sudo modprobe usb_storage
```

🔎 **Vérification :**

```
lsmod | grep usb_storage
```

Résultat :

```
usb_storage        73728  1
```

🧪 Effet réel après rechargement

👉 Rebranche la clé USB :

- ✅ Détection immédiate
- ✅ Montage automatique
- ✅ Visible dans le gestionnaire de fichiers

🔍 Étape 4 : Vérifier les messages du noyau

```
dmesg | tail
```

Exemple de messages :

```
usbcore: registered new interface driver usb_storage
```

❓ Pourquoi on n'a PAS besoin de connaître tous les modules

🔢 Il y a énormément de modules

Un noyau Linux standard contient :

- **des milliers de modules**
- dépendants du matériel, du réseau, des systèmes de fichiers, etc.

📌 Exemple :

```
ls /lib/modules/$(uname -r)/kernel | wc -l
```

👉 Impossible (et inutile) de tous les mémoriser.

🧠 La philosophie Linux (très importante)

Sous Linux, on ne demande pas :

« Quels sont tous les modules ? »

Mais plutôt :

« Quel module ai-je besoin dans cette situation précise ? »

✅ Ce qu'il FAUT vraiment connaître (l'essentiel)

1☐ Le concept de module

✔ À quoi sert un module
✔ Différence noyau monolithique / modulaire
✔ Chargement dynamique (sans redémarrage)

2☐ Les commandes clés (indispensables)

Objectif	Commande
Voir les modules chargés	`lsmod`
Charger un module	`modprobe`
Décharger un module	`modprobe -r`
Infos sur un module	`modinfo`
Messages noyau	`dmesg`

👉 **Ça, oui, il faut les maîtriser.**

3☐ Quelques modules courants (exemples)

Pas tous, mais **les plus fréquents** :

🔌 Matériel

- `usb_storage` → clés USB
- `usbhid` → clavier / souris USB

💽 Stockage

- `loop` → fichiers montés comme disques
- `ext4` → système de fichiers

🌐 Réseau

- `e1000`, `rtl8139` → cartes réseau (VM)
- `bridge`, `bonding`

📌 Les connaître **par usage**, pas par cœur.

4☐ Savoir chercher un module (compétence clé)

Exemple :

« Ma carte réseau ne fonctionne pas »

Tu fais :

```
lspci
lsusb
modprobe
dmesg
```

👉 **La compétence, c'est le raisonnement**, pas la mémorisation.

🧠 Analogie pédagogique (qui marche très bien)

🧰 **Modules Linux = outils dans une boîte**

- Tu n'apprends pas **tous les outils**
- Tu apprends :
  - à **lire l'étiquette**
  - à **choisir le bon outil**
  - à **l'utiliser correctement**

📌 Conclusion courte

✔ Comprendre le principe
✔ Maîtriser les commandes
✔ Connaître quelques modules courants
✔ Savoir chercher et diagnostiquer

❌ Mémoriser tous les modules

🔧 1. Module `usb_storage` — Clés USB / disques externes

📌 Rôle

Permet la détection et l'utilisation des **périphériques de stockage USB**.

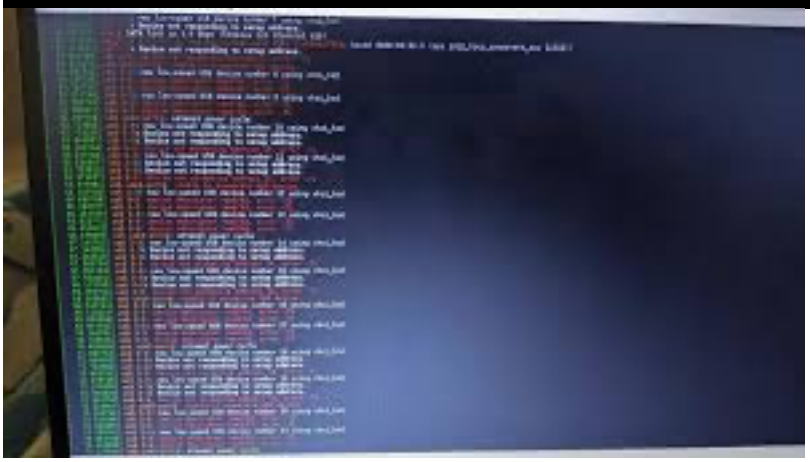
❌ Problème courant

- Clé USB branchée mais **non détectée**
- Rien ne s'affiche dans le gestionnaire de fichiers

🔍 Diagnostic

```
lsmod | grep usb_storage
dmesg | tail
```





🛠️ Solution

```
sudo modprobe usb_storage
```

Vérification :

```
lsmod | grep usb_storage
```

💿 2. Module `loop` — Images disque / Snap

📌 Rôle

Monte des **fichiers comme disques virtuels** (ISO, Snap, VM).

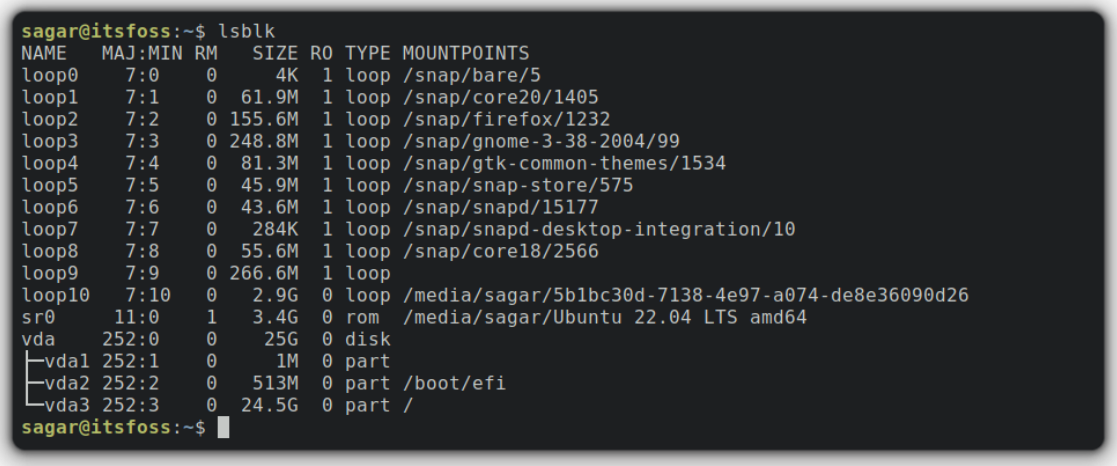
❌ Problème courant

- ISO ne se monte pas
- Applications Snap ne démarrent pas

🔍 Diagnostic

```
lsmod | grep loop
lsblk
```

```
bob@suse1:~> lsmod
Module                  Size  Used by
nls_utf8               12557  1
lp                     17752  0
nfsd                  335706  9
lockd                  93786  1 nfsd
nfs_acl                12837  1 nfsd
auth_rpcgss            53231  1 nfsd
sunrpc                261329  17 nfsd,lockd,nfs_acl,auth_rpcgss
md5                    12627  0
iscsi_trgt            120554  4
crc32c                 12759  0
snd_pcm_oss            62203  0
snd_mixer_oss          22423  1 snd_pcm_oss
snd_seq_midi           13324  0
snd_seq_midi_event     14899  1 snd_seq_midi
snd_seq                69824  2 snd_seq_midi,snd_seq_midi_event
edd                    14544  0
af_packet              30965  0
mperf                  12667  0
vmhgfs                 72145  0
vsock                  60957  0
acpiphp                28117  0
quota_v2               13402  2
```



🛠️ Solution

```
sudo modprobe loop
```

🌐 3. Module `e1000` / `rtl8139` — Carte réseau (VM)

📌 Rôle

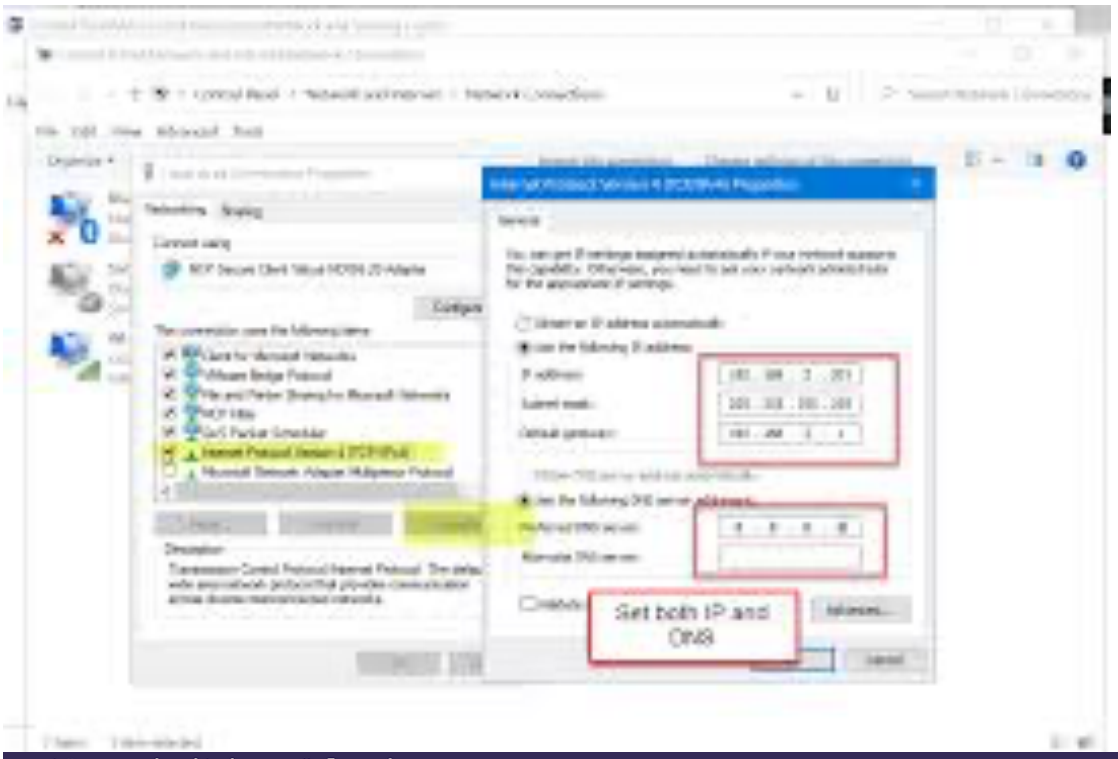
Pilotes réseau très utilisés dans **VirtualBox / VMware**.

❌ Problème courant

- Pas d'Internet
- Interface réseau absente

🔍 Diagnostic

```
ip a
lspci | grep -i ethernet
```




```
root@newredswitches:~# lspci
00:00.0 Host bridge: Intel Corporation 440FX - 82441FX PMC [Natoma] (rev 02)
00:01.0 ISA bridge: Intel Corporation 82371SB PIIX3 ISA [Natoma/Triton II]
00:01.1 IDE interface: Intel Corporation 82371SB PIIX3 IDE [Natoma/Triton II]
00:01.2 USB controller: Intel Corporation 82371SB PIIX3 USB [Natoma/Triton II] (rev 01)
00:01.3 Bridge: Intel Corporation 82371AB/EB/MB PIIX4 ACPI (rev 03)
00:02.0 VGA compatible controller: Device 1234:1111 (rev 02)
00:03.0 Unclassified device [00ff]: Red Hat, Inc. Virtio memory balloon
00:05.0 PCI bridge: Red Hat, Inc. QEMU PCI-PCI bridge
00:12.0 Ethernet controller: Red Hat, Inc. Virtio network device
00:1e.0 PCI bridge: Red Hat, Inc. QEMU PCI-PCI bridge
00:1f.0 PCI bridge: Red Hat, Inc. QEMU PCI-PCI bridge
01:01.0 SCSI storage controller: Red Hat, Inc. Virtio SCSI
```

🛠️ Solution

```
sudo modprobe e1000
```

ou

```
sudo modprobe rtl8139cp
```

📁 4. Module `ext4` — Système de fichiers

📌 Rôle

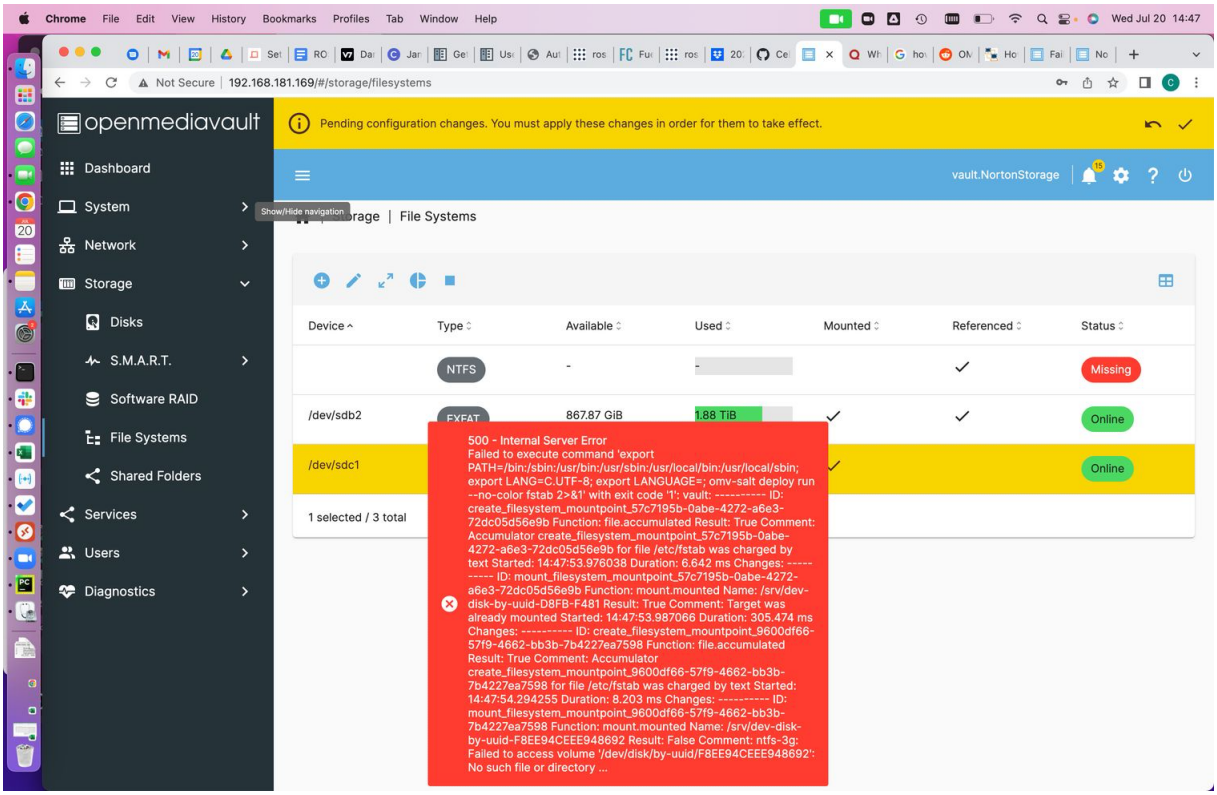
Gère le **système de fichiers Linux le plus courant**.

❌ Problème courant

- Partition ne se monte pas
- Erreur lors du démarrage

🔍 Diagnostic

```
lsblk -f
dmesg | grep ext4
```



🛠️ Solution

```
sudo fsck.ext4 /dev/sdXN
```

⚠️ partition démontée uniquement

🔊 5. Module snd_hda_intel — Son

📌 Rôle

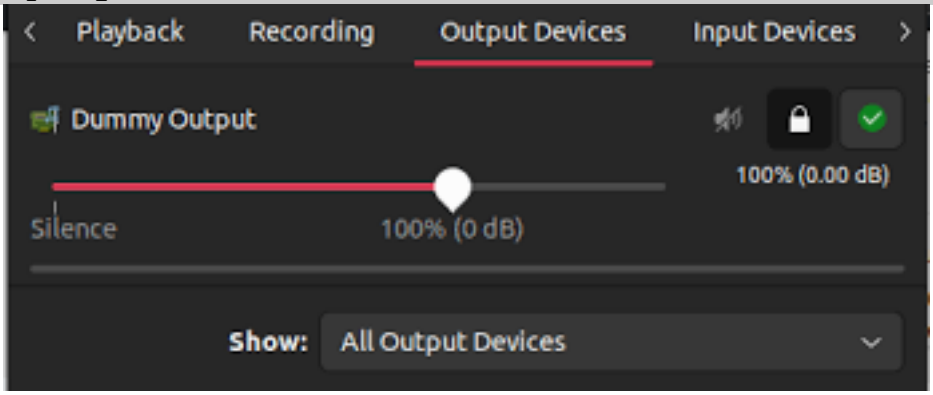
Pilote audio principal sur PC et VM.

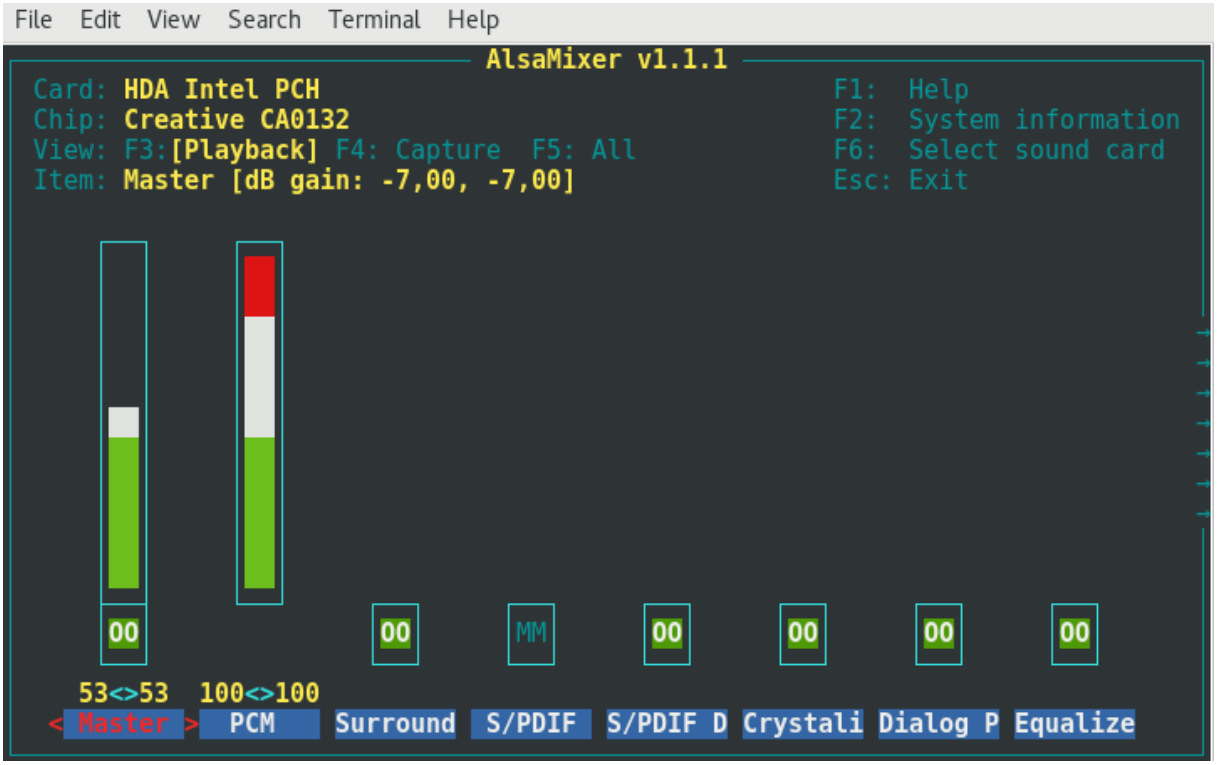
❌ Problème courant

- Pas de son
- Périphérique audio absent

🔍 Diagnostic

```
lsmod | grep snd
aplay -l
```





🛠️ Solution

```
sudo modprobe snd_hda_intel
```

Puis redémarrage du son :

```
pulseaudio -k
```

🔐 6. Module vfat — Clés USB FAT32

📌 Rôle

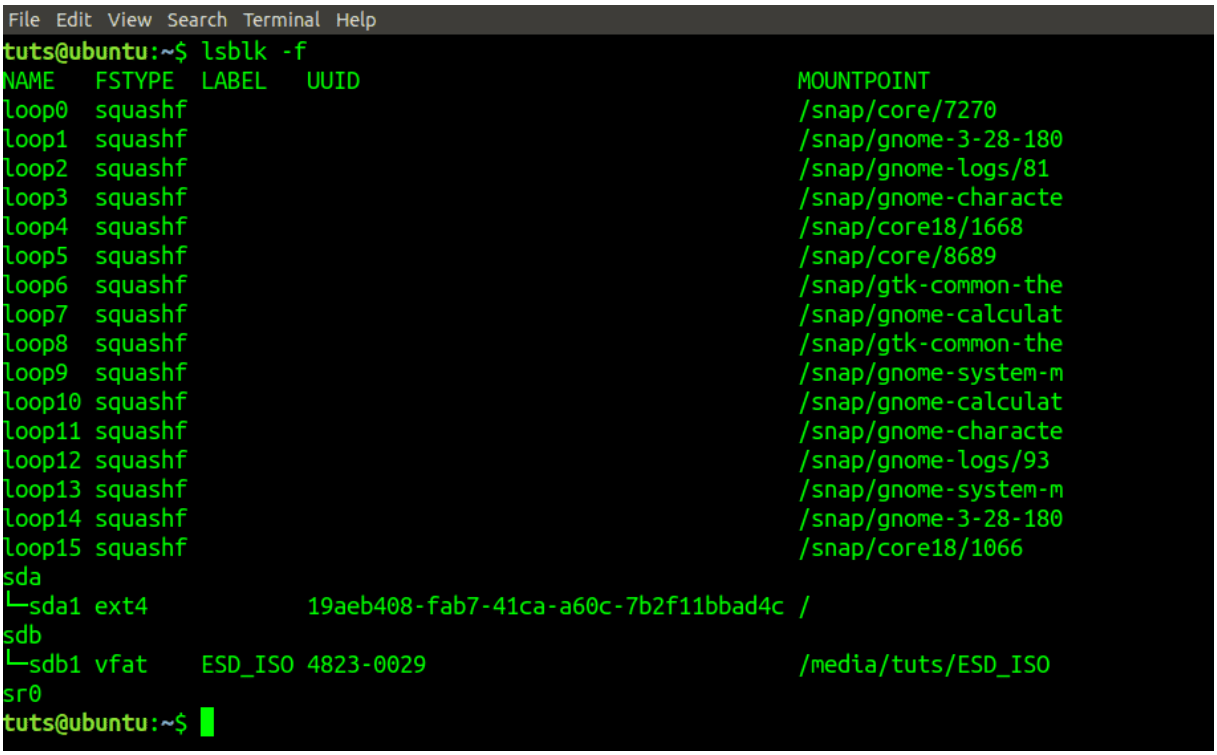
Permet la lecture/écriture **FAT32**.

❌ Problème courant

- Clé USB détectée mais non montée

🔍 Diagnostic

```
lsblk -f
mount | grep vfat
```



🛠️ Solution

```
sudo modprobe vfat
sudo mount /dev/sdXN /mnt
```

📌 Tableau récapitulatif

Module	Fonction	Problème typique
usb_storage	Stockage USB	Clé non détectée
loop	ISO / Snap	ISO non montable
e1000	Réseau VM	Pas d'Internet
ext4	Fichiers Linux	Partition non montée
snd_hda_intel	Son	Audio absent
vfat	FAT32	Clé non lisible

🎓 Conclusion

La maîtrise des modules du noyau Linux repose non pas sur leur mémorisation exhaustive, mais sur la capacité à diagnostiquer un dysfonctionnement à l'aide des commandes système et à charger dynamiquement le module approprié.
